

Backtracking & Divide and Conquer

అన్ని అవకాశాలని క్రమపద్ధతిలో వెతకడం – choose, explore, un-choose.

ప్రతి problem కీ: ఏ **pattern** ఇది → ఎందుకు ఆ pattern → dry run → optimal JavaScript code → complexity → edge cases. [DSA_Patterns_Telugu.pdf](#)
pattern-first దృష్టి; ఈ file ఆ patterns ని నిజమైన LeetCode problems మీద అమలు చేస్తుంది.

ఈ document చదివిన తర్వాత Backtracking మరియు Divide-and-Conquer problems మళ్ళీ నిన్ను భయపెట్టవు. Backtracking అంటే మెదడులో ఒక **decision tree** గీసి, ప్రతి node దగ్గర "choose → explore → un-choose" చేస్తూ అన్ని దారులూ తిరగడం. అది ఒకసారి కళ్ళకి కట్టినట్టు అర్థమైతే – permutations, combinations, subsets, N-Queens, maze, word search – అన్నీ **ఒకే template** తో పడిపోతాయి. ప్రతి problem కి: ఎలా ఆలోచించాలి (decision tree ని text లో గీస్తాం), naive నుండి optimal వరకు, pruning ఎలా చేయాలి, clean commented JavaScript, పెన్సిల్ తో గీసినట్టు dry run, complexity (tree లో ఎన్ని nodes?), pattern takeaway, మరియు సాధారణ తప్పులు (un-choose మర్చిపోవడం, duplicates) – అన్నీ ఉంటాయి.

లక్ష్యం: DSA అస్సలు తెలియని person ని – ఎవరికైతే recursion అంటే ఏమిటో అర్థమవుతుందో కానీ "అన్ని possibilities generate చేయడం" ఎలాగో తెలియదో – వాళ్ళని SSE (Senior Software Engineer) interview లో ఈ 11 problems confident గా solve చేసేలా తయారు చేయడం. మనం facts బట్టి పట్టడం కాదు – **ఎలా ఆలోచించాలో** నేర్చుకుంటాం. Backtracking template ఒకటి, Divide-and-Conquer template ఒకటి – ఈ రెండు పట్టుకుంటే, ఇక్కడ ఉన్న 11 మాత్రమే కాదు, వీటిలాంటి 100 problems కూడా solve చేయగలవు.

గమనిక: Recursion అంటే ఏమిటి, call stack ఎలా పని చేస్తుంది, Big-O notation, time/space complexity, tree/linked-list basics – ఇలాంటి పునాదులు (fundamentals)

[DSA_00_Foundations_Telugu.md](#) లో ఉన్నాయి. Backtracking ముఖ్యంగా recursion మీద ఆధారపడుతుంది – కాబట్టి recursion సరిగ్గా అర్థం కాకపోతే ముందు అది చదువు. ఇక్కడ మనం నేరుగా Backtracking & Divide-and-Conquer patterns లోకి దిగుతాం. Solutions అన్నీ **JavaScript (ES2020+)** లో.

విషయ సూచిక (Table of Contents)

Pattern Primers (ముందు వీటిని చదువు – problems కి foundation)

- Pattern: Backtracking (choose → explore → un-choose; decision tree; pruning)
- Pattern: Divide and Conquer (split → solve subproblems → combine; recursion tree)

Part 1 – Backtracking Problems

1. Letter Combinations of a Phone Number (LeetCode #17) – Medium
2. Combinations (LeetCode #77) – Medium
3. Permutations (LeetCode #46) – Medium
4. Combination Sum (LeetCode #39) – Medium
5. N-Queens II (LeetCode #52) – Hard
6. Generate Parentheses (LeetCode #22) – Medium
7. Word Search (LeetCode #79) – Medium

Part 2 – Divide and Conquer Problems

8. Convert Sorted Array to Binary Search Tree (LeetCode #108) – Easy
9. Sort List (LeetCode #148) – Medium
10. Construct Quad Tree (LeetCode #427) – Medium
11. Merge k Sorted Lists (LeetCode #23) – Hard

Pattern: Backtracking

వివరణ

Backtracking అంటే – ఒక సమస్యకి అన్ని సాధ్యమైన solutions (లేదా ఒక valid solution) కావాలంటే, మనం ఒక్కో decision తీసుకుంటూ ముందుకు వెళ్తాం; ఆ దారి fail అయితే, వెనక్కి వచ్చి (backtrack), ఇండాక తీసుకున్న decision ని రద్దు చేసి, వేరే option try చేస్తాం. అంటే. ఇది brute-force కి ఒక organized రూపం – కానీ smart గా, dead-end లు ముందే గుర్తించి కత్తిరిస్తాం (pruning).

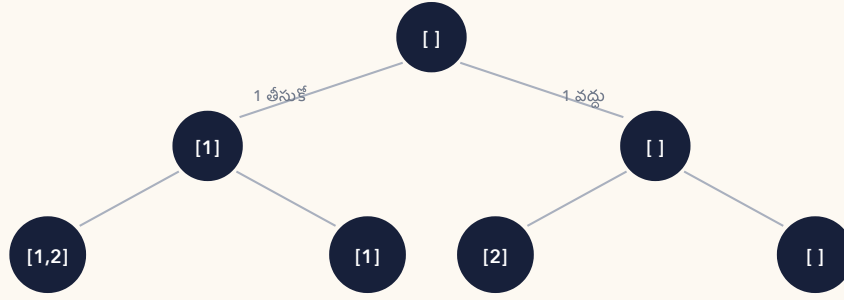
మెదడులో ఒక **decision tree** ఊహించుకో. Root దగ్గర మొదలవుతాం. ప్రతి node దగ్గర, "ఇప్పుడు ఏ options ఉన్నాయి?" అని చూస్తాం. ఒక్కో option = tree లో ఒక **branch (edge)**. ఒక branch ఎంచుకుని కిందికి దిగుతాం (explore). Leaf (చివరి node) కి చేరితే – ఒక complete candidate తయారైంది; అది valid అయితే answer లో పెడతాం. తర్వాత వెనక్కి వచ్చి, ఇండాక ఎంచుకున్న branch ని వదిలేసి, పక్క branch లోకి దిగుతాం. ఇలా tree లోని ప్రతి దారి systematic గా తిరుగుతాం.

మూడు మంత్రాలు – ఇవి backtracking యొక్క అత్త:

1. **Choose (ఎంచుకో):** ఒక option ని ప్రస్తుత `path` లో పెట్టు. (ఉదా: `path.push(x)`)
2. **Explore (లోతుకి వెళ్ళు):** ఆ choice తీసుకున్నాక recurse – తర్వాతి decision కి వెళ్ళు.
3. **Un-choose / Backtrack (రద్దు చేయి):** recurse తిరిగి వచ్చాక, ఆ option ని తీసేయి (`path.pop()`) – తద్వారా `path` మళ్ళీ శుభ్రమై, పక్క branch కోసం సిద్ధమవుతుంది.

ఈ మూడేది – **un-choose** – backtracking లో అత్యంత కీలకం మరియు అందరూ మర్చిపోయేది. ఇది మర్చిపోతే, ఒక branch లో పెట్టిన choices పక్క branch లోకి leak అయి, తప్పు answers వస్తాయి.

Pruning (కత్తిరించడం): కొన్ని branches ఎటువంటి valid solution ఇవ్వవని ముందే తెలిస్తే, అక్కడే ఆగిపో – ఆ subtree మొత్తం explore చేయకు. ఇదే backtracking ని plain brute-force కన్నా వేగం చేస్తుంది. ఉదా: Combination Sum లో remaining sum negative అయిపోతే, ఇక ముందుకు వెళ్ళి లాభం లేదు – return.



మూడు అడుగుల నృత్యం – ఇదే మొత్తం backtracking

CHOOSE – `path.push(candidate)`

EXPLORE – `backtrack(next)`

UN-CHOOSE – `path.pop()` ← ఇదే "backtrack"

చివరి అడుగు మర్చిపోతే శాఖలన్నీ ఒకదానితో ఒకటి కలుషితమవుతాయి

ఫలితంలో చేర్చబట్టావు `copy` తీసుకోవాలి – `result.push([...path])`. Reference పంపితే, తర్వాత `path` మారినప్పుడు ఇప్పటికే చేర్చిన జవాబులూ మారిపోతాయి. ఇది అత్యంత సాధారణమైన bug.

Real-life Scenario

ఒక చీకటి maze (దారితప్పించే labyrinth) లో ఉన్నావని ఊహించుకో. బయటికి వెళ్ళే దారి వెతుకుతున్నావు. చేతిలో ఒక సుద్దముక్క (chalk) ఉంది. ఒక junction వచ్చింది – మూడు దారులు. ఎడమ దారిలోకి వెళ్ళావు, గోడ మీద గుర్తు పెడతావు (choose). కొంతదూరం వెళ్ళాక dead-end (గోడ) వచ్చింది. ఇప్పుడు ఏం చేస్తావు? వెనక్కి junction దాకా నడిచొచ్చి, ఇందాక పెట్టిన గుర్తు చెరిపేసి (un-choose), మధ్య దారి try చేస్తావు. మళ్ళీ dead-end అయితే, కుడి దారి. ఇలా ప్రతి junction దగ్గర అన్ని దారులూ, systematic గా, గుర్తు పెడుతూ-చెరుపుతూ try చేస్తే – బయటికి వెళ్ళే దారి కచ్చితంగా దొరుకుతుంది (ఉంటే). ఆ "గుర్తు పెట్టడం-చెరపడం" నే **choose-unchoose**. ఆ maze నే **decision tree**.

ఇంకో analogy: **సూట్ కేస్ combination lock**. 4 digits, ఒక్కటి 0-9. నీకు code మర్చిపోయింది. ఏం చేస్తావు? మొదటి digit 0 fix చేసి, రెండో digit 0, మూడో 0, నాలుగో 0 → 0000 try. Fail. చివరి digit 1 కి మార్పు → 0001. ... 0009 దాకా వెళ్ళి, అప్పుడు మూడో digit 1 కి మార్పు, చివరిది మళ్ళీ 0 నుండి. ఇది సరిగ్గా backtracking – ప్రతి "position" ఒక decision, ప్రతి digit ఒక choice, ఒక combination fail అయితే వెనక్కిచ్చి తర్వాతిది.

ఎలా గుర్తించాలి (Recognition Signals)

ఈ signals కనిపిస్తే backtracking గురించి ఆలోచించు:

- **"అన్ని (all) generate చెయ్యి / return చెయ్యి"** – "all permutations", "all combinations", "all subsets", "all valid ...". Answer ఒక్కటి కాదు, **collection** కావాలి.
- **"ఎన్ని ways / ఎన్ని solutions"** – count కావాలి (N-Queens II లాంటిది). అన్ని possibilities enumerate చేసి count చేస్తాం.
- **Permutations / Combinations / Subsets / Partitions** – ఈ మాటలు నేరుగా backtracking ని సూచిస్తాయి.
- **Board / Grid problems** – N-Queens, Sudoku, Word Search, maze. ప్రతి cell దగ్గర choice, dead-end అయితే వెనక్కి.

- **"Build a string/sequence step by step, constraints తో"** – Generate Parentheses, phone letter combinations. ఒక్కో character ఒక decision.
- **Exponential search space** – $\text{choices}^{\text{depth}}$. Input size చిన్నది ($n \leq 15-20$ లాంటిది) అయితే, exponential solution intended అని hint. DP తో overlap ఉంటే optimize చేయొచ్చు, కానీ "అన్ని enumerate చెయ్యి" అంటే backtracking.

Template Code (JavaScript)

ఇదే backtracking యొక్క **canonical skeleton** – ప్రతి problem దీన్ని కొంచెం మార్చి వాడతాం. బట్టీ కొట్టు:

```
function solve(input) {
  const result = []; // final answers ఇక్కడ పోగవుతాయి
  const path = []; // ప్రస్తుతం build చేస్తున్న partial candidate

  function backtrack(/* state: index / start / remaining ... */) {
    // 1) BASE CASE – candidate పూర్తయిందా? (goal reached)
    if (/* candidate complete */) {
      result.push([...path]); // ⚠️ COPY చెయ్యి ([...path]) – reference కాదు!
      return;
    }

    // 2) ప్రస్తుత decision కి ఉన్న అన్ని CHOICES మీద loop
    for (const choice of /* available choices */) {
      // (optional) PRUNING – ఈ choice ఎటూ fail అయితే skip
      if (/* invalid */) continue;

      path.push(choice); // CHOOSE (గుర్తు పెట్టు)
      backtrack(/* next state */); // EXPLORE (లోతుకి వెళ్ళు)
      path.pop(); // UN-CHOOSE (గుర్తు చెరిపేయి) ← మర్చిపోకు!
    }
  }

  backtrack(/* initial state */);
  return result;
}
```

మూడు గుర్తుంచుకోవాల్సిన విషయాలు: (1) **base case** ఎప్పుడు candidate పూర్తి; (2) leaf దగ్గర `[...path]` గా **copy** పెట్టడం (లేకపోతే అన్ని answers ఒకే array ని point చేసి, చివర్లో అన్నీ ఖాళీగా ఉంటాయి); (3) `push` కి సరిగ్గా జతగా `pop` – **choose ఉన్నచోట un-choose ఉండాలి**.

Complexity

Backtracking complexity ని **decision tree లో ఎన్ని nodes ఉన్నాయి** అని లెక్కిస్తాం – ఎందుకంటే ప్రతి node ఒక recursion call.

- **Branching factor (b):** ప్రతి node దగ్గర ఎన్ని choices. **Depth (d):** tree ఎంత లోతు.
- **Nodes $\approx O(b^d)$** – exponential. ఉదా: permutations $\rightarrow n!$ leaves, subsets $\rightarrow 2^n$ leaves.
- **ప్రతి leaf దగ్గర** candidate ని copy చేయడానికి $O(\text{length})$ time పడితే, total = $O(\text{nodes} \times \text{copy-cost})$.
- **Space:** recursion depth $O(d)$ call stack కి, plus `path` $O(d)$, plus result store చేయడానికి output size. Output ని సాధారణంగా space లో లెక్కించరు (అది కావాల్సిందే).

- **Pruning** worst-case Big-O ని మార్చకపోవచ్చు, కానీ practical గా చాలా వేగం చేస్తుంది (అనవసర subtrees కట్టిరించడం వల్ల).

1. Letter Combinations of a Phone Number (LeetCode #17) – Medium

సమస్య: పాత mobile keypad లో ప్రతి digit కి కొన్ని letters ఉంటాయి (2 → abc, 3 → def, ...). 2 - 9 digits ఉన్న string `digits` ఇస్తారు. ఈ digits type చేస్తే వచ్చే అన్ని సాధ్యమైన letter combinations return చెయ్యాలి (ఏ order లో అయినా). 1, 0 కి letters లేవు, అవి రావు.

Keypad mapping: 2 → abc, 3 → def, 4 → ghi, 5 → jkl, 6 → mno, 7 → pqrs, 8 → tuv, 9 → wxyz.

Constraints: $0 \leq \text{digits.length} \leq 4$; digits లో 2 - 9 మాత్రమే. `digits` ఖాళీ అయితే ఖాళీ array `[]` return చెయ్యి.

ఉదాహరణ:

```
Input: digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Input: digits = ""    → Output: []
Input: digits = "2"   → Output: ["a","b","c"]
```

ఎలా ఆలోచించాలి:

"23" చూడు. మొదటి digit 2 → {a, b, c}. రెండో digit 3 → {d, e, f}. మనకి కావాల్సింది: మొదటి set నుండి ఒక letter, రెండో set నుండి ఒక letter – అన్ని జతలు. అంటే 2 కి 3 choices, ఒక్కో choice కి 3 కి 3 choices → $3 \times 3 = 9$ combinations.

ఇది సరిగ్గా ఒక **decision tree**. Root దగ్గర digit 2 కి 3 branches (a/b/c). ప్రతి branch కింద, digit 3 కి 3 branches (d/e/f). Leaf కి చేరితే – 2 letters పోగయ్యాయి, ఒక complete combination.



కాబట్టి: ప్రస్తుత **digit position** ఒక "decision". ఆ position కి ఉన్న letters మీద loop, ఒక్కోటి `path` లో పెట్టి (choose), తర్వాతి position కి recurse (explore), తిరిగొచ్చాక తీసేయి (un-choose). Position `digits.length` కి చేరితే – `path` నిండింది, ఒక answer.

Optimal Approach:

ఇది classic backtracking. Brute-force అంటే nested loops – కానీ digits count fixed కాదు (0 నుండి 4), కాబట్టి hard-coded nested loops రాయలేం. Recursion తో generalize చేస్తాం.

Plan:

1. Digit → letters mapping ని object లో పెట్టు.
2. `backtrack(index)` : `index` = ప్రస్తుతం ఏ digit position లో ఉన్నాం.
3. Base case: `index === digits.length` → `path` పూర్తి → `result` లో push చేసి return.
4. లేకపోతే: ప్రస్తుత digit `digits[index]` కి ఉన్న letters మీద loop – choose/explore/un-choose.
5. Edge case: `digits` ఖాళీ అయితే వెంటనే `[]` return (లేకపోతే `[""]` వచ్చేస్తుంది – తప్పు).

Solution (JavaScript):

```
function letterCombinations(digits) {
  if (digits.length === 0) return []; // ఖాళీ input → ఖాళీ answer (కీలక edge case)

  const map = {
    "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
    "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz",
  };

  const result = [];
  const path = []; // ప్రస్తుతం build చేస్తున్న letters

  function backtrack(index) {
    // BASE CASE: అన్ని digits కి letter ఎంచుకున్నాం
    if (index === digits.length) {
      result.push(path.join("")); // path array ని string గా మార్చి పెట్టు
      return;
    }

    const letters = map[digits[index]]; // ఈ digit కి available letters
    for (const ch of letters) {
      path.push(ch); // CHOOSE
      backtrack(index + 1); // EXPLORE – తర్వాతి digit
      path.pop(); // UN-CHOOSE
    }
  }

  backtrack(0);
  return result;
}
```

Dry Run: `digits = "23"`

```
backtrack(0): digit '2' → "abc"
  ch='a': path=[a] → backtrack(1): digit '3' → "def"
    ch='d': path=[a,d] → backtrack(2): index==2==len → push "ad" ✓; pop → path=[a]
    ch='e': path=[a,e] → backtrack(2): push "ae" ✓; pop → path=[a]
    ch='f': path=[a,f] → backtrack(2): push "af" ✓; pop → path=[a]
  pop → path=[]
  ch='b': path=[b] → backtrack(1): → "bd","be","bf"; pop → path=[]
```

```
ch='c': path=[c] → backtrack(1): → "cd","ce","cf"; pop → path=[]
result = ["ad","ae","af","bd","be","bf","cd","ce","cf"] ✓
```

గమనించు: ప్రతి `push` తర్వాత సరిగ్గా `pop` – అందుకే `a` branch అయిపోయాక `path` శుభ్రమై `b` కి సిద్ధమవుతుంది.

Complexity:

- **Time:** $O(4^n \times n)$ – ఇక్కడ $n = \text{digits.length}$. ప్రతి digit కి ఎక్కువలో ఎక్కువ 4 letters (7, 9 కి), కాబట్టి leaves 4^n వరకు. ప్రతి leaf దగ్గర `path.join("")` కి $O(n)$. $n \leq 4$ కాబట్టి practical గా చాలా చిన్నది.
- **Space:** $O(n)$ – recursion depth n , `path` size n . (Output ని వేరుగా లెక్కిస్తే $4^n \times n$.)

గుర్తుంచుకోవాల్సినది:

ఇది "ప్రతి position కి ఒక set నుండి ఒకటి ఎంచుకో" అనే backtracking యొక్క పునాది pattern (Cartesian product). ప్రతి digit ఒక decision level, ప్రతి letter ఒక branch. ఇదే idea: **all combinations of independent choice-sets** – ఉదా "ఒక menu లో starter/main/dessert ప్రతిదాంట్లో ఒకటి ఎంచుకుని అన్ని meals generate చెయ్యి". Template ని అలానే వాడు.

సాధారణ తప్పులు:

- **ఖాళీ input edge case:** `digits = ""` కి `if (digits.length === 0) return []` పెట్టకపోతే, `backtrack(0)` వెంటనే base case hit అయి `result = [""]` (ఖాళీ string ఉన్న array) return చేస్తుంది – expected `[]` కి తప్పు.
- **Un-choose మర్చిపోవడం:** `path.pop()` లేకపోతే, `a` branch letters `b` branch లోకి leak అయి "abd" లాంటి చెత్త వస్తుంది.
- **Leaf దగ్గర copy మర్చిపోవడం:** ఇక్కడ `path.join("")` కొత్త string ఇస్తుంది కాబట్టి safe. కానీ arrays store చేసే problems లో `[...path]` గా copy తప్పనిసరి – లేకపోతే అన్ని answers ఒకే mutable array ని point చేస్తాయి.
- **0 / 1 handle చేయాలా అని కంగారు:** constraints ప్రకారం అవి రావు – map లో పెట్టాల్సిన అవసరం లేదు.

2. Combinations (LeetCode #77) – Medium

సమస్య: రెండు integers n మరియు k ఇస్తారు. 1 నుండి n వరకు ఉన్న numbers లో నుండి k numbers ఎంచుకునే అన్ని combinations return చెయ్యాలి. Combination అంటే **order పట్టింపు లేదు** – `[1,2]` మరియు `[2,1]` ఒకటే, ఒకసారే రావాలి.

Constraints: $1 \leq n \leq 20$, $1 \leq k \leq n$.

ఉదాహరణ:

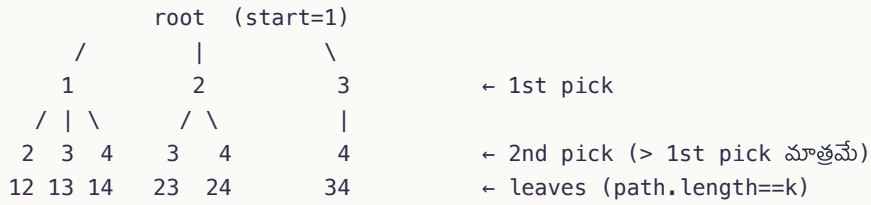
```
Input: n = 4, k = 2
Output: [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]
```

Input: $n = 1, k = 1 \rightarrow$ Output: $[[1]]$

ఎలా ఆలోచించాలి:

$[1, 2] = [2, 1]$ కాబట్టి, duplicates రాకుండా ఒక trick వాడతాం: **ఎప్పుడూ పెరుగుతున్న order లోనే** numbers ఎంచుకుంటాం. అంటే ఒక number i ఎంచుకున్నాక, తర్వాత i కన్నా పెద్ద numbers మాత్రమే చూస్తాం. దీనివల్ల $[1, 2]$ వస్తుంది కానీ $[2, 1]$ రాదు (2 తర్వాత 1 చూడం). ఇదే start pointer యొక్క magic.

Decision tree ($n=4, k=2$): root దగ్గర మొదటి number 1/2/3 (4 కాదు – 4 ఎంచుకుంటే రెండేది కావాలి కానీ 4 తర్వాత ఏమీ లేదు). ఒక్కో choice కింద, దాని కన్నా పెద్దవి.



path లో k numbers నిండగానే – leaf, ఒక combination. లేకపోతే start నుండి n వరకు loop, ఒక్కోటి choose చేసి $i+1$ నుండి recurse.

Pruning (కీలకో): ఇంకా $k - \text{path.length}$ numbers కావాలి. కానీ i నుండి n వరకు అంత numbers లేకపోతే – ఆ branch వృథా. కాబట్టి loop ని $i \leq n - (k - \text{path.length}) + 1$ వరకే తిప్పు. ఉదా: $k=2$, path ఖాళీ, $n=4 \rightarrow i \leq 4 - 2 + 1 = 3$ ($i=4$ skip; ఎందుకంటే 4 తర్వాత రెండేది లేదు). ఇది చాలా subtrees కత్తిరిస్తుంది.

Solution (JavaScript):

```
function combine(n, k) {
  const result = [];
  const path = [];

  function backtrack(start) {
    // BASE CASE: k numbers ఎంచుకున్నాం
    if (path.length === k) {
      result.push([...path]); // ⚠️ COPY – reference కాదు
      return;
    }

    // PRUNING: ఇంకా (k - path.length) కావాలి; అన్ని numbers ఉంటేనే loop
    const need = k - path.length;
    const last = n - need + 1; // ఇంతకన్నా పైన మొదలుపెడితే k నిండదు
    for (let i = start; i <= last; i++) {
      path.push(i); // CHOOSE
      backtrack(i + 1); // EXPLORE – i కన్నా పెద్దవి మాత్రమే (i+1 నుండి)
      path.pop(); // UN-CHOOSE
    }
  }

  backtrack(1);
}
```

```

    return result;
}

```

Dry Run: `n = 4, k = 2`

```

backtrack(1): need=2, last=4-2+1=3 → i: 1..3
  i=1: path=[1] → backtrack(2): need=1, last=4 → i:2..4
    i=2: path=[1,2] → len==2 → push [1,2]✓; pop
    i=3: path=[1,3] → push [1,3]✓; pop
    i=4: path=[1,4] → push [1,4]✓; pop
    pop → path=[]
  i=2: path=[2] → backtrack(3): need=1, last=4 → i:3..4
    i=3: push [2,3]✓ ; i=4: push [2,4]✓ ; pop → path=[]
  i=3: path=[3] → backtrack(4): need=1, last=4 → i:4
    i=4: push [3,4]✓ ; pop → path=[]
result = [[1,2], [1,3], [1,4], [2,3], [2,4], [3,4]] ✓

```

గమనించు: `i=4` root level లో try చేయలేదు – pruning వల్ల (last=3). అదే optimization.

Complexity:

- **Time:** $O(C(n,k) \times k)$ – total combinations $C(n,k)$, ఒక్కోదాన్ని copy చేయడానికి $O(k)$. Tree లో internal nodes కూడా ఉంటాయి కానీ leaves dominate.
- **Space:** $O(k)$ – recursion depth `k`, `path` size `k`. (Output $C(n,k) \times k$ వేరు.)

గుర్తుంచుకోవాల్సినది:

start pointer = "order పట్టింపు లేని combinations లో duplicates ఆపే master trick". ఒక element ఎంచుకున్నాక `i+1` నుండి recurse చేస్తే, ఎప్పుడూ increasing order → `[2,1]` లాంటి permutation-duplicate రాదు. ఇదే idea: **Subsets** (base case లేకుండా ప్రతి node దగ్గర push), **Combination Sum**, **Subsets II**. Pruning formula `n - need + 1` ని అర్థం చేసుకో – interview లో "ఇంకా optimize చేయగలవా?" అంటే ఇదే చెప్పాలి.

సాధారణ తప్పులు:

- **backtrack(i) బదులు backtrack(i + 1) :** `i` పంపితే అదే number మళ్ళీ ఎంచుకోవచ్చు (`[1,1]`) – ఇది Combination Sum behavior, ఇక్కడ తప్పు. Combinations కి `i+1` తప్పనిసరి.
- **start బదులు ఎప్పుడూ 1 నుండి loop:** అప్పుడు `[1,2]` మరియు `[2,1]` రెండూ వస్తాయి – duplicates.
- **Copy మర్చిపోవడం:** `result.push(path)` (copy లేకుండా) పెడితే, అన్ని entries ఒకే array reference – చివర్లో అన్నీ `[] . [...path]` వాడు.
- **Pruning లేకపోయినా answer correct** – కానీ interview లో pruning చూపిస్తే senior signal.

COMBINATIONS VS PERMUTATIONS - START VS USED[]

COMBINATIONS vs PERMUTATIONS - ఒక్క తేడా

Combinations - క్రమం ముఖ్యం కాదు

```
backtrack(start, path)
for (i = start; i < n; i++)
    backtrack(i + 1, ...)
→ [1,2] మరియు [2,1] ఒకటే
```

Permutations - క్రమం ముఖ్యం

```
backtrack(path, used[])
for (i = 0; i < n; i++)
    if (used[i]) continue
→ [1,2] మరియు [2,1] వేరు
```

ఒక్క పంక్తి తేడా - కానీ అర్థం పూర్తిగా వేరు

start వాడతే - వెనక్కి చూడము, కాబట్టి duplicates రావు (combinations).
used[] వాడతే - అన్నీ చూస్తూం కానీ ఒక్కసారే వాడతాం (permutations).
Duplicates ఉన్న input అయితే - sort చేసి, $i > \text{start} \ \&\& \ \text{nums}[i] == \text{nums}[i-1] \rightarrow \text{skip}$.

3. Permutations (LeetCode #46) - Medium

సమస్య: Distinct integers array `nums` ఇస్తారు. దానిలోని elements యొక్క **అన్ని permutations** (అన్ని అమరికలు) return చెయ్యాలి. Permutation లో **order పట్టించు ఉంది** - `[1,2]` మరియు `[2,1]` రెండూ వేరు, రెండూ కావాలి.

Constraints: $1 \leq \text{nums.length} \leq 6$; అన్ని elements distinct.

ఉదాహరణ:

```
Input: nums = [1,2,3]
Output: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]

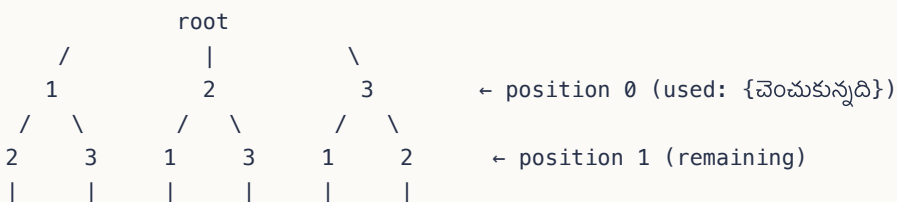
Input: nums = [0,1] → Output: [[0,1], [1,0]]
```

ఎలా ఆలోచించాలి:

Combinations లో order పట్టించు లేదు కాబట్టి **start** తో increasing order maintain చేశాం. కానీ permutations లో **order పట్టించు ఉంది** - `[1,2,3]` మరియు `[3,2,1]` రెండూ కావాలి. కాబట్టి ప్రతి position దగ్గర, **ఇంకా వాడని ఏ number నైనా** ఎంచుకోవచ్చు - increasing constraint లేదు.

కానీ ఒక permutation లో ఒక number **ఒకసారే** రావాలి. కాబట్టి "ఏ numbers ఇప్పటికే వాడాను?" అని track చేయాలి - దీనికి `used[]` boolean array. ఒక number choose చేసేటప్పుడు `used[i]=true`, un-choose చేసేటప్పుడు `used[i]=false`.

Decision tree (`nums=[1,2,3]`): root దగ్గర మొదటి position కి 3 choices (1/2/3). ఒక్కోదాని కింద మిగతా 2, తర్వాత చివరి 1.



3	2	3	1	2	1	← position 2 (last remaining)
123	132	213	231	312	321	← leaves (3! = 6)

`path.length === nums.length` → అన్ని positions నిండాలు → ఒక permutation.

Optimal Approach:

Backtracking with a `used[]` visited-tracker. Plan:

1. `used` = అన్నీ false తో array.
2. `backtrack()` : `path` నిండితే (`length == n`) → copy చేసి push, return.
3. లేకపోతే: అన్ని indices `i` మీద loop. `used[i]` అయితే skip. లేకపోతే – `used[i]=true`, push, recurse, pop, `used[i]=false`.

Solution (JavaScript):

```
function permute(nums) {
  const result = [];
  const path = [];
  const used = new Array(nums.length).fill(false); // ఏ index వాడమో track

  function backtrack() {
    // BASE CASE: అన్ని positions నిండాలు
    if (path.length === nums.length) {
      result.push([...path]); // ⚠️ COPY
      return;
    }

    for (let i = 0; i < nums.length; i++) {
      if (used[i]) continue; // ఇప్పటికీ ఈ permutation లో వాడం → skip
      used[i] = true; // CHOOSE
      path.push(nums[i]);
      backtrack(); // EXPLORE – తర్వాతి position
      path.pop(); // UN-CHOOSE
      used[i] = false;
    }
  }

  backtrack();
  return result;
}
```

Dry Run: `nums = [1,2,3]` (మొదటి branch మాత్రమే వివరంగా)

```
backtrack(): path=[], used=[F,F,F]
i=0(1): used=[T,F,F] path=[1] → backtrack()
  i=1(2): used=[T,T,F] path=[1,2] → backtrack()
    i=2(3): used=[T,T,T] path=[1,2,3] → len==3 → push [1,2,3]✓
      pop, used[2]=F → path=[1,2]
    pop, used[1]=F → path=[1]
  i=2(3): used=[T,F,T] path=[1,3] → backtrack()
    i=1(2): path=[1,3,2] → push [1,3,2]✓ ; backtrack ...
      pop, used[2]=F → path=[1]
```

```

pop,used[0]=F → path=[]
i=1(2): ... → [2,1,3],[2,3,1]
i=2(3): ... → [3,1,2],[3,2,1]
result = [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]] ✓ (6 = 3!)

```

Complexity:

- **Time:** $O(n! \times n)$ – $n!$ permutations, ఒక్కోదాన్ని copy చేయడానికి $O(n)$. Tree లో మొత్తం nodes $\approx n \cdot (n!)$ కూడా అదే bound.
- **Space:** $O(n)$ – recursion depth n , `path` n , `used` n . (Output $n! \times n$ వేరు.)

గుర్తుంచుకోవాల్సినది:

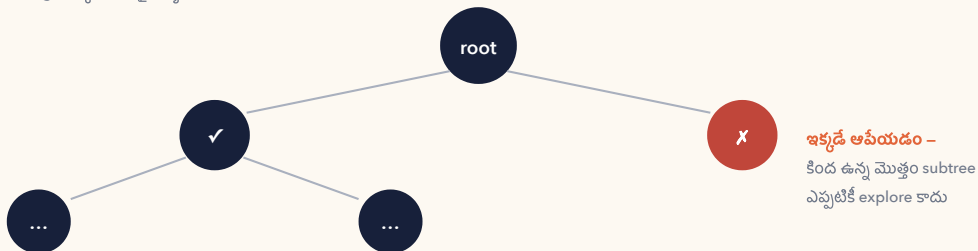
Combinations vs Permutations తేడా = start vs used. Order పట్టింపు లేకపోతే (combinations/subsets) → `start` pointer, ఎప్పుడూ ముందుకే. Order పట్టింపు ఉంటే (permutations) → `used[]`, ప్రతిసారి అన్ని indices నుండి ఎంచుకో, వాడినవి skip. ఈ ఒక్క తేడా అర్థమైతే, ఈ రెండు families మొత్తం నీ చేతిలో. Duplicates ఉన్న version (Permutations II) కి: sort చేసి, `if (i>0 && nums[i]==nums[i-1] && !used[i-1]) continue;` అనే skip condition జోడిస్తాం.

సాధారణ తప్పులు:

- **`used[i]=false` (un-mark) మర్చిపోవడం:** పక్క branch కి వెళ్ళేటప్పుడు ఆ index permanently used గా ఉండిపోయి, permutations missing అవుతాయి. `path.pop()` తో పాటు `used[i]=false` తప్పనిసరి – రెండూ un-choose లో భాగం.
- **`start` వాడటం:** permutations కి `start` వాడితే combinations వస్తాయి (order lost) – తప్పు.
- **Copy మర్చిపోవడం:** మామూలే – `[...path]`.
- **Duplicates ఉన్న input:** ఈ problem distinct guarantee ఇస్తుంది; distinct కాకపోతే ఈ code duplicate permutations ఇస్తుంది. అప్పుడు Permutations II approach కావాలి.

PRUNING · శాఖలని ముందే కత్తిరించడం

PRUNING – backtracking యొక్క అసలు నైపుణ్యం



Pruning వ్యాహాలు

Sort చేసి early break ($\text{sum} > \text{target}$)
 Constraint ని ముందే check (N-Queens)
 ఇప్పటివరకటి ఉత్తమం కంటే చెడ్డదైతే ఆపడం

ప్రభావం

సిద్ధాంతంలో complexity మారదు – $O(n!)$ అలానే
 కానీ ఆచరణలో 100 రెట్లు వేగం కావచ్చు
 N-Queens 8×8 – pruning లేకపోతే అనాధ్యం

4. Combination Sum (LeetCode #39) – Medium

సమస్య: Distinct integers array `candidates` మరియు ఒక `target` ఇస్తారు. `candidates` నుండి numbers ఎంచుకుని sum సరిగ్గా `target` వచ్చే అన్ని **unique combinations** return చెయ్యాలి. ఒక **number** ని ఎన్నిసార్లైనా వాడొచ్చు (unlimited reuse). రెండు combinations లో ఒకే numbers ఒకే counts లో ఉంటే అవి same (order పట్టించు లేదు).

Constraints: `1 <= candidates.length <= 30`; అన్నీ distinct; `2 <= candidates[i] <= 40`; `1 <= target <= 40`. Guaranteed: answers count < 150.

ఉదాహరణ:

```
Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]           (2+2+3=7, 7=7)

Input: candidates = [2,3,5], target = 8
Output: [[2,2,2,2],[2,3,3],[3,5]]
```

ఎలా ఆలోచించాలి:

ఇది Combinations కి పెద్ద మేనమామ – రెండు మార్పులతో:

1. **Reuse allowed** – ఒక number ని మళ్ళీ మళ్ళీ వాడొచ్చు. కాబట్టి choose చేశాక `i+1` కాదు, `i` నుండి recurse (అదే number మళ్ళీ available).
2. **Target constraint** – `path` నిండాలి అని కాదు, `sum == target` అయినప్పుడు answer.

Order పట్టించు లేదు (`[2,2,3] = [3,2,2]`) కాబట్టి duplicates అపడానికి మళ్ళీ **start pointer** – ఎప్పుడూ ముందుకే (index తగ్గించు), కానీ అదే index మళ్ళీ వాడొచ్చు కాబట్టి `i` నుండి (`i+1` కాదు).

Decision tree (candidates=[2,3,6,7], target=7): "remaining" (target - so-far-sum) ని track చేద్దాం.

```

              remaining=7 (start=0)
            /2      |3      \6, \7
    rem=5(2..) rem=4(3..) rem=1 rem=0 → [7]✓
    /2 \3      \3
  rem=3 rem=2 rem=1
  /2  |3      x(3>2 skip via pruning)
rem=1 rem=0→[2,2,3]✓
  x
```

Pruning (కీలకం): `candidates` ని sort చేస్తే, ఒక candidate `remaining` కన్నా పెద్దదైతే – దాన్ని, దాని తర్వాతి అన్నిటినీ (ఇంకా పెద్దవి) skip చేయొచ్చు (`break`). Negative remaining లోకి వెళ్ళే అవసరమే లేదు.

Solution (JavaScript):

```
function combinationSum(candidates, target) {
  const result = [];
  const path = [];
  candidates.sort((a, b) => a - b); // pruning కోసం ascending
```

```

function backtrack(start, remaining) {
  // BASE CASE 1: సరిగ్గా target కి చేరాం
  if (remaining === 0) {
    result.push([...path]);
    return;
  }

  for (let i = start; i < candidates.length; i++) {
    // PRUNING: sorted కాబట్టి ఇది పెద్దదైతే, తర్వాతివన్నీ పెద్దవే → ఆపేయి
    if (candidates[i] > remaining) break;

    path.push(candidates[i]); // CHOOSE
    // ⚠ i (i+1 కాదు) – అదే number మళ్ళీ వాడొచ్చు (reuse)
    backtrack(i, remaining - candidates[i]); // EXPLORE
    path.pop(); // UN-CHOOSE
  }
}

backtrack(0, target);
return result;
}

```

Dry Run: candidates = [2,3,6,7] (sorted అలానే), target = 7

```

backtrack(0, 7):
  i=0 (2): path=[2] → backtrack(0, 5):
    i=0 (2): path=[2,2] → backtrack(0, 3):
      i=0 (2): path=[2,2,2] → backtrack(0, 1):
        i=0 (2): 2>1 → break (pruning) → dead end
        pop → [2,2]
      i=1 (3): path=[2,2,3] → backtrack(1, 0): rem==0 → push [2,2,3]✓; pop →
[2,2]
        pop → [2]
      i=1 (3): path=[2,3] → backtrack(1, 2): i=1(3): 3>2 break → dead; pop → [2]
        pop → []
      i=1 (3): path=[3] → backtrack(1, 4): ... 3+3=6≠7, 3+ anything>7 → no; pop
      i=2 (6): path=[6] → backtrack(2, 1): 6>1 break; pop
      i=3 (7): path=[7] → backtrack(3, 0): rem==0 → push [7]✓; pop
result = [[2,2,3],[7]] ✓

```

Complexity:

- **Time:** $O(N^{(T/M)})$ roughly – N = candidates count, T = target, M = smallest candidate. Tree depth $\leq T/M$ (చిన్న candidate తో ఎన్నిసార్లు target కి చేరతాం), branching $\leq N$. ఒక్కో valid path copy $O(T/M)$. Exact bound tricky, కానీ exponential.
- **Space:** $O(T/M)$ – recursion depth (worst case అన్నీ smallest candidate).

గుర్తుంచుకోవాల్సినది:

"Reuse allowed" = backtrack(i, ...) ; **"reuse కాదు" = backtrack(i+1, ...)** . ఈ ఒక్క అక్షరం (i vs $i+1$) తేడాతో Combinations family మొత్తం control చేయొచ్చు. Sort + break pruning ఒక powerful combo – sorted array లో "ఇది పెద్దదైతే తర్వాతివన్నీ పెద్దవే" అనే logic చాలా backtracking

problems లో పనికొస్తుంది. **Combination Sum II** (#40): reuse లేదు (`i+1`) + duplicates skip (`if (i>start && candidates[i]==candidates[i-1]) continue`).

సాధారణ తప్పులు:

- **i+1 వాడేయడం:** అప్పుడు reuse పోతుంది – `[2,2,3]` రాదు (2 ఒకసారే). Reuse కావాలంటే `i`.
- **start లేకుండా 0 నుండి loop:** `[2,2,3]` , `[2,3,2]` , `[3,2,2]` – permutation duplicates వస్తాయి. `start` తో ఆపు.
- **Pruning లేకుండా remaining negative:** `if (candidates[i] > remaining) break` (లేదా `continue` unsorted అయితే) పెట్టకపోతే, negative remaining లోకి దిగి infinite-ish waste. Base case ని `remaining < 0 → return` గా కూడా రాయొచ్చు, కానీ `sort+break` cleaner.
- **Copy:** `[...path]` మామూలే.

5. N-Queens II (LeetCode #52) – Hard

సమస్య: `n × n` chessboard మీద `n` queens ని – **ఏ రెండు queens ఒకరినొకరు attack చేయకుండా** – పెట్టే **విధానాల సంఖ్యను (count)** return చెయ్యాలి. Chess లో queen అడ్డంగా (row), నిలువుగా (column), మరియు రెండు diagonals లో ఎంత దూరమైనా attack చేస్తుంది. (N-Queens I actual boards అడుగుతుంది; ఇది కేవలం ఎన్ని అని.)

Constraints: `1 <= n <= 9`.

ఉదాహరణ:

Input: `n = 4` → Output: 2 (4×4 కి 2 valid arrangements)
Input: `n = 1` → Output: 1

ఎలా ఆలోచించాలి:

కీలక observation: **ప్రతి row లో సరిగ్గా ఒక్క queen ఉండాలి** (`n` queens, `n` rows, రెండు ఒకే row లో ఉంటే attack). కాబట్టి సమస్యని ఇలా మార్చుకో: **row 0 కి ఒక column ఎంచుకో, row 1 కి ఒక column, ... row n-1 దాకా** – ప్రతి row ఒక decision, ఆ row లో ఏ column లో queen పెట్టాలి అనేది choice.

అంటే decision tree: depth = `n` (ఒక్కో row ఒక level), branching = `n` (ఒక్కో column). కానీ ప్రతి placement దగ్గర **ముందు పెట్టిన queens తో clash అవుతుందా** అని check చేసి, clash అయితే ఆ branch skip (pruning).

Attack ఎప్పుడు?

- **Same column:** ఇప్పటి column ముందే వాడామా.
- **Same "+" diagonal:** ఈ diagonal మీద అన్ని cells కి `row - col` constant. (ఉదా `(0,0),(1,1),(2,2)` అన్నిటికీ `row-col=0`.)
- **Same "/" diagonal:** అన్ని cells కి `row + col` constant. (ఉదా `(0,2),(1,1),(2,0)` అన్నిటికీ `row+col=2`.)

కాబట్టి మూడు Set లు: cols (వాడిన columns), diag1 (వాడిన row-col), diag2 (వాడిన row+col). ఒక cell (row,col) safe గా ఉందా అంటే – ఈ మూడింటిలో ఏదీ ఇప్పటికే occupied కాకూడదు. ఇది **O(1) check** – board మొత్తం scan చేయనవసరం లేదు!

```
row 0: col=0? col=1? col=2? col=3? ← 4 choices, ఒక్కటి try
      (col ఎంచుకున్నాక cols/diag1/diag2 లో mark చేసి row 1 కి)
row 1: safe columns మాత్రమే
...
row n: అన్ని rows నిండాయి → ఒక valid board → count++
```

Optimal Approach:

Backtracking row-by-row, O(1) conflict check తో. Plan:

1. cols, diag1 (row-col), diag2 (row+col) – మూడు Set .
2. backtrack(row) : row === n → n queens placed → count++ , return.
3. లేకపోతే: col 0..n-1 మీద loop. cols / diag1 / diag2 లో conflict ఉంటే skip. లేకపోతే: మూడు sets లో add (choose), backtrack(row+1) (explore), మూడు sets నుండి delete (un-choose).

Solution (JavaScript):

```
function totalNQueens(n) {
  let count = 0;
  const cols = new Set(); // వాడిన columns
  const diag1 = new Set(); // వాడిన (row - col) → "\" diagonals
  const diag2 = new Set(); // వాడిన (row + col) → "/" diagonals

  function backtrack(row) {
    // BASE CASE: అన్ని rows లో queen పెట్టాం → ఒక valid arrangement
    if (row === n) {
      count++;
      return;
    }

    for (let col = 0; col < n; col++) {
      // PRUNING: ఈ cell attack అవుతుందా? (O(1))
      if (cols.has(col) || diag1.has(row - col) || diag2.has(row + col)) {
        continue; // clash → ఈ column skip
      }

      // CHOOSE – ఈ cell లో queen పెట్టు
      cols.add(col);
      diag1.add(row - col);
      diag2.add(row + col);

      backtrack(row + 1); // EXPLORE – తర్వాతి row

      // UN-CHOOSE – queen తీసేయి (మూడు sets నుండి)
      cols.delete(col);
      diag1.delete(row - col);
      diag2.delete(row + col);
    }
  }
}
```

```

    backtrack(0);
    return count;
}

```

Dry Run: `n = 4` (ఒక valid path + backtrack చూద్దాం)

```

row0: col=0 place → sets: cols{0} d1{0} d2{0}
row1: col=0? cols has 0 → skip. col=1? d1 has 1-1=0 → skip. col=2 place → cols{0,2}
d1{0,-1} d2{0,3}
row2: col=0 skip(cols). col=1? d2 has 2+1=3 → skip. col=2 skip. col=3? d1 has 2-
3=-1 → skip. → no col! dead-end, backtrack
row1 undo col=2 ; col=3 place → cols{0,3} d1{0,-2} d2{0,4}
row2: col=1 place → ...
row3: col=2 place → row==4 → count++ ✓ (board: (0,0)(1,3)(2,?)... one
solution family)
... col=0 branch తర్వాత col=1,2,3 branches try
అన్ని branches తిరిగి → count = 2 ✓

```

Complexity:

- **Time:** $O(n!)$ roughly – row 0 కి n choices, row 1 కి $1 \leq n-1$ safe (column repeat కాదు), ... కాబట్టి $n!$ కన్నా తక్కువ (diagonals మరింత కత్తిరిస్తాయి). Set operations $O(1)$.
- **Space:** $O(n)$ – recursion depth n , మూడు sets ఒక్కోటి $\leq n$ elements.

గుర్తుంచుకోవాల్సినది:

Diagonal ని `row-col` / `row+col` constant గా encode చేయడం – ఇది board/grid backtracking లో బంగారు trick. Board అంతా scan చేసి "attack అవుతుందా" చూడాల్సిన అవసరం లేదు – మూడు Set lookups చాలు, $O(1)$. ఇదే **"state ని smart గా encode చేసి pruning ని $O(1)$ చేయడం"** – Sudoku solver లో row/col/box sets కూడా అదే idea. "ప్రతి row/level కి ఒక్క placement" అనే observation తో 2D problem ని 1D decision sequence గా మార్చడం కూడా గుర్తుంచుకో.

సాధారణ తప్పులు:

- **Un-choose లో మూడు sets నుండి delete చేయకపోవడం:** `cols.delete` మాత్రమే చేసి `diag1/diag2` మర్చిపోతే – ఆ diagonals permanently blocked గా ఉండి, valid boards miss అవుతాయి. Choose లో 3 add → un-choose లో 3 delete, జతగా.
- **Diagonal formula తారుమారు:** `row-col` మరియు `row+col` ని కలిపేయడం. `"` (top-left→bottom-right) కి `row-col` constant; `/` (top-right→bottom-left) కి `row+col` constant. `row-col` negative కావొచ్చు (Set కి problem లేదు).
- **$O(1)$ check బదులు board scan:** 2D board పెట్టి ప్రతిసారి row/col/diag scan చేస్తే correct కానీ నెమ్మది; interview లో sets approach expected.
- **N-Queens I తో గుండరగోళం:** ఇది count మాత్రమే (`count++`). Boards కావాలంటే `path` maintain చేసి strings build చేయాలి – logic అదే.

6. Generate Parentheses (LeetCode #22) – Medium

సమస్య: n j/pairs of parentheses ఇస్తే, అన్ని **well-formed (valid, balanced)** parentheses combinations generate చెయ్యాలి. Well-formed అంటే ప్రతి (కి సరిగ్గా జతగా), సరైన order లో (ఎ prefix లోనూ) count (count ని దాటకూడదు).

Constraints: $1 \leq n \leq 8$.

ఉదాహరణ:

```
Input: n = 3
Output: ["((()))","(()())","(())()","()()()","()()()"]

Input: n = 1 → Output: ["()"]
```

ఎలా ఆలోచించాలి:

Total $2n$ characters – ఒక్కటి (లేదా) . Naive గా అన్ని $2^{(2n)}$ combinations generate చేసి, ఒక్కొక్కటి "valid ఆ?" అని check చేయించు – కానీ చాలా waste (చాలావి invalid).

Smart గా ఆలోచిద్దాం: **invalid అయ్యే బ్రాంచ్‌ని ముందే కత్తిరిద్దాం (pruning)**. ఒక valid string build చేస్తున్నప్పుడు, ఏ position లోనైనా (పెట్టవచ్చా,) పెట్టవచ్చా అనేది రెండు rules తో తెలుస్తుంది:

- (**పెట్టొచ్చు** – ఇప్పటిదాకా వాడిన open brackets n కన్నా తక్కువ ఉంటే ($open < n$). ఇంకా open మిగిలుంది.
-) **పెట్టొచ్చు** – ఇప్పటిదాకా closed brackets, opened కన్నా తక్కువ ఉంటే ($close < open$). అంటే మూయడానికి ఒక unmatched (**ఉంది**.

ఈ రెండు constraints మాత్రమే పాటిస్తే, generate అయ్యే **ప్రతి string valid** – invalid ని అస్సలు build చేయమే. ఇదే "constraint-guided construction" – backtracking యొక్క అందం.

Decision tree (n=2): ప్రతి node దగ్గర రెండు choices (/) – కానీ pruning తో కొన్ని మాత్రమే allowed.

```

""" (open=0,close=0)
    | ( (open<2 ✓      [close<open? 0<0 x]
"" (" (1,0)
    / ( ✓          \ ) (close<open:0<1 ✓)
""((" (2,0)       ""(" (1,1)
    |              | ( ✓
""((" (2,1)       ""(" (2,1)
    |              |)
""((" (2,1)       ""(())" ✓
    |              |)
""(())" ✓         ""(())" ✓

```

`string.length === 2n` → balanced string పూర్తి → answer.

Optimal Approach:

Backtracking with two counters (`open` , `close`) as the pruning state. Plan:

1. `backtrack(open, close)` : ఇప్పటిదాకా ఎన్ని `( మరియు )` వాడాలి.
2. Base case: `path.length === 2n` (లేదా `open === n && close === n`) → push, return.
3. `open < n` → `(` add చేసి recurse (choose/explore/un-choose).
4. `close < open` → `)` add చేసి recurse.

Solution (JavaScript):

```
function generateParenthesis(n) {
  const result = [];
  const path = []; // ప్రస్తుత string ని array గా (join వేగం కోసం)

  function backtrack(open, close) {
    // BASE CASE: 2n characters నిండాలు → balanced string
    if (path.length === 2 * n) {
      result.push(path.join(""));
      return;
    }

    // OPTION 1: '(' పెట్టొచ్చా? - ఇంకా open bracket మిగిలుంటే
    if (open < n) {
      path.push("(");
      backtrack(open + 1, close); // open ఒకటి పెరిగింది
      path.pop();                // UN-CHOOSE
    }

    // OPTION 2: ')' పెట్టొచ్చా? - మూయడానికి unmatched '(' ఉంటే
    if (close < open) {
      path.push(")");
      backtrack(open, close + 1); // close ఒకటి పెరిగింది
      path.pop();                // UN-CHOOSE
    }
  }

  backtrack(0, 0);
  return result;
}
```

Dry Run: `n = 2`

```
backtrack(0,0): len 0
'(' (open0<2): path=[(] → backtrack(1,0):
  '(' (open1<2): path=[(] → backtrack(2,0):
    '(' ? open2<2 ✗ ; ')' close0<open2 ✓: path=[()] → backtrack(2,1):
      ')' close1<open2 ✓: path=[()] → len4 → push "()" ✓; pop
      pop → [(]
      pop → [(]
    ')' (close0<open1 ✓): path=[()] → backtrack(1,1):
      '(' (open1<2): path=[()] → backtrack(2,1):
        ')' close1<2 ✓: path=[()] → push "()" ✓; pop → [(]
        pop → [(]
      ')' ? close1<open1 ✗
      pop → [(]
      pop → []
  result = ["()", "()"] ✓
```

గమనించు: invalid strings (")) " , ") (") అస్సలు generate అవలేదు – pruning వాటిని ఎప్పుడూ create చేయనీయలేదు.

Complexity:

- **Time:** $O(4^n / \sqrt{n})$ – ఇది **n-th Catalan number** $C(n)$. Valid parentheses strings count Catalan number, ఒక్కోదాన్ని build చేయడం $O(n)$. అది asymptotically $4^n / (n \sqrt{n}) \times n \approx 4^n / \sqrt{n}$.
- **Space:** $O(n)$ – recursion depth $2n$, path $2n$. (Output Catalan $\times n$ వేరు.)

గుర్తుంచుకోవాల్సినది:

ఇది "**constraint-guided generation**" – invalid ని generate-then-filter చేయకుండా, **validity rules** నే **branching conditions** గా వాడి, **valid మాత్రమే build చేయడం**. open < n మరియు close < open – ఈ రెండు invariants ముందే enforce చేయడం వల్ల "check valid" step అవసరమే లేదు. ఇదే idea ఉన్న problems: **Restore IP Addresses, valid sequences with rules**. Catalan number pattern (balanced structures – parentheses, BSTs, mountains) కూడా గుర్తుంచుకో.

సాధారణ తప్పులు:

- close < open బదులు close < n : అప్పుడు ") (" లాంటి invalid strings వస్తాయి –) ని unmatched (లేకుండా పెట్టేస్తుంది. Rule: మూయడానికి తెరిచింది ఉండాలి → close < open .
- open <= n (< బదులు <=): (ని n కన్నా ఎక్కువ పెట్టేసి, balance చెడుతుంది.
- Two if లు కాకుండా if/else : (పెట్టగలిగినప్పుడు else వల్ల) branch skip అవుతుంది – చాలా valid strings miss. రెండూ విడి if లు (else కాదు) – ఎందుకంటే రెండు options ఒకేసారి possible కావచ్చు.
- Un-choose మర్చిపోవడం: ప్రతి push తర్వాత pop . లేకపోతే path leak.

7. Word Search (LeetCode #79) – Medium

సమస్య: $m \times n$ grid board (characters తో) మరియు ఒక word ఇస్తారు. word ని board లో **adjacent cells** (పై, కింద, ఎడమ, కుడి – diagonal కాదు) ని వరుసగా కలుపుతూ construct చేయగలమా అని true / false return చెయ్యాలి. **ఒకే cell ని ఒక word లో రెండుసార్లు వాడకూడదు.**

Constraints: $1 \leq m, n \leq 6$; $1 \leq \text{word.length} \leq 15$; అన్నీ lowercase/uppercase English letters.

ఉదాహరణ:

```
board = [
  ["A","B","C","E"],
  ["S","F","C","S"],
  ["A","D","E","E"]
]
Input: word = "ABCCED" → Output: true
Input: word = "SEE" → Output: true
Input: word = "ABCB" → Output: false (B ని రెండుసార్లు వాడాలి – కుదరదు)
```

ఎలా ఆలోచించాలి:

`word` ని build చేయడం అంటే – ఒక cell నుండి మొదలుపెట్టి, పక్క cells కి కదులుతూ, ఒక్క character match చేస్తూ వెళ్ళడం. ఒక దారి fail అయితే (అక్షరం match కాకపోతే / అంచు దాటితే / cell మళ్ళీ వాడాల్సి వస్తే), వెనక్కి వచ్చి వేరే దిశ try చేయడం – ఇదే maze backtracking!

Word ఏ cell నుండైనా మొదలవ్వచ్చు కాబట్టి, **ప్రతి cell ని starting point గా** try చేస్తాం (rows × cols outer loops). ఒక cell (r,c) నుండి `backtrack(r, c, index) - index = word లో ఏ character కోసం చూస్తున్నాం.`

ప్రతి cell దగ్గర 4 choices (4 directions). Decision tree branching = 4, depth = word.length.

```
start (r,c) matches word[0]?
  → up:    (r-1,c) matches word[1]? → up/down/left/right...
  → down:  (r+1,c) matches word[1]? → ...
  → left:  (r,c-1) ...
  → right: (r,c+1) ...
```

"Cell మళ్ళీ వాడకూడదు" ఎలా enforce? – visited track చేయాలి. Extra `visited` array బదులు, ఒక clean trick: ప్రస్తుత cell ని temporarily ఒక **sentinel** (`"#"`) తో replace చేసి, recurse అయ్యాక **తిరిగి original character** పెట్టేయడం. ఇదే un-choose. దీనివల్ల ఆ path లో ఈ cell మళ్ళీ match కాదు (`"#"` ఏ letter కీ equal కాదు), కానీ backtrack అయ్యాక వేరే paths కి available.

Optimal Approach:

DFS backtracking from every cell, with in-place marking. Plan:

- Outer double loop: ప్రతి (r,c) ని start గా `backtrack(r,c,0)` ; ఏదైనా `true` ఇస్తే return `true`.
- `backtrack(r,c,index)` :
 - Base success: `index === word.length` → అన్ని characters matched → `true`.
 - Fail (bounds/mismatch): r,c range దాటితే లేదా `board[r][c] !== word[index]` → `false`.
 - Mark: `board[r][c] = "#"` (choose/visit).
 - 4 directions లో recurse – ఏదైనా `true` ఇస్తే found.
 - Restore: `board[r][c] = original` (un-choose).

Solution (JavaScript):

```
function exist(board, word) {
  const rows = board.length, cols = board[0].length;

  function backtrack(r, c, index) {
    // BASE SUCCESS: అన్ని characters match అయ్యాయి
    if (index === word.length) return true;

    // FAIL: అంచు దాటింది / ఇప్పటికే visited / అక్షరం match కాలేదు
    if (r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] !== word[index]) {
      return false;
    }
  }
}
```

```

const temp = board[r][c];
board[r][c] = "#"; // CHOOSE - ఈ cell ని "visited" గా mark (sentinel)

// EXPLORE - 4 directions; ఏదైనా ఒకటి true ఇస్తే చాలు (short-circuit ||)
const found =
  backtrack(r + 1, c, index + 1) || // down
  backtrack(r - 1, c, index + 1) || // up
  backtrack(r, c + 1, index + 1) || // right
  backtrack(r, c - 1, index + 1); // left

board[r][c] = temp; // UN-CHOOSE - cell ని restore (వేరే paths కి available)
return found;
}

// ప్రతి cell ని starting point గా try చెయ్యి
for (let r = 0; r < rows; r++) {
  for (let c = 0; c < cols; c++) {
    if (backtrack(r, c, 0)) return true;
  }
}
return false;
}

```

Dry Run: word = "SEE", board పైనది. Start వెతుకుతూ (2,3)='E'? word[0]='S' కాదు... (1,3)='S' match!

```

backtrack(1,3,0): board[1][3]='S'==word[0]='S' ✓ → mark '#'
  down (2,3,1): 'E'==word[1]='E' ✓ → mark '#'
    down (3,3): out of bounds → false
    up (1,3): '#' != word[2]='E' → false
    right (2,4): out of bounds → false
    left (2,2,2): 'E'==word[2]='E' ✓ → mark '#'
      index+1=3 == word.length → return TRUE ✓
    found = true → restore (2,2)='E'
  found = true → restore (2,3)='E'
found = true → restore (1,3)='S'
→ exist returns true ✓

```

Complexity:

- **Time:** $O(m \times n \times 4^L)$ – $L = \text{word.length}$. ప్రతి starting cell ($m \times n$) నుండి, ప్రతి step 4 directions (నిజానికి మొదటి step తర్వాత 3, ఎందుకంటే వెనక్కి వెళ్ళలేం), depth $L \rightarrow 4^L$. Worst case exponential.
- **Space:** $O(L)$ – recursion depth L (call stack). In-place marking వల్ల extra visited array అవసరం లేదు.

గుర్తుంచుకోవాల్సినది:

Grid DFS backtracking – "in-place marking" trick. Visited ని extra array లో కాకుండా, cell ని temporarily మార్చి, backtrack అయ్యాక restore చేయడం – space ఆదా. ఇదే pattern: **Number of Islands** (flood fill), **Word Search II** (Trie తో optimize), **maze/path problems**. "4 (లేదా 8) directions

లో కదులు + visited + un-mark" అనేది board-DFS యొక్క ఆత్మ. || short-circuit వల్ల ఒక direction success అయితే మిగతావి try చేయదు – వేగం.

సాధారణ తప్పులు:

- **Restore (un-choose) మర్చిపోవడం:** `board[r][c] = temp` లేకపోతే, ఆ cell శాశ్వతంగా "#" గా ఉండిపోయి, వేరే starting points / paths దాన్ని వాడలేక, valid words కూడా false అంటాయి. అతి సాధారణ bug.
- **Bounds check ముందు board access:** `board[r][c]` ని bounds check కన్నా ముందు చదివితే undefined / crash. ఎప్పుడూ `r<0 || r>=rows || ...` ముందు, `board[r][c]` తర్వాత (|| short-circuit protect చేస్తుంది).
- **Base case order:** `index === word.length` success check ని **mismatch check కన్నా ముందు** పెట్టాలి – లేకపోతే word పూర్తయ్యాక కూడా `board[r][c] !== word[index]` (out-of-range index) చూసి crash/wrong.
- **Diagonal moves:** problem 4 directions మాత్రమే; diagonals కలిపితే తప్పు.
- **Same cell reuse:** marking లేకపోతే "ABCB" కి B ని రెండుసార్లు వాడి తప్పుగా true అంటుంది.

Pattern: Divide and Conquer

వివరణ

Divide and Conquer అంటే – ఒక పెద్ద సమస్యని **చిన్న చిన్న ఒకేలాంటి subproblems** గా విడగొట్టి (**divide**), ఒక్కో subproblem ని విడిగా solve చేసి (conquer), ఆ solutions ని **కలిపి (combine)** పెద్ద answer తయారుచేయడం. మూడు దశలు:

1. **Divide (విడగొట్టు):** Input ని రెండు (లేదా అంతకంటే ఎక్కువ) చిన్న భాగాలుగా చీల్చు. సాధారణంగా సగం-సగం.
2. **Conquer (జయించు):** ప్రతి భాగాన్నీ **recursively** solve చెయ్యి. భాగం చాలా చిన్నదైతే (base case – ఒక element లేదా ఖాళీ), నేరుగా answer తెలుసు.
3. **Combine (కలుపు):** Subproblems solutions ని కలిపి original problem answer తయారుచెయ్యి. ఈ "కలపడం" లోనే అసలు పని ఉంటుంది (ఉదా: merge sort లో రెండు sorted halves ని merge చేయడం).

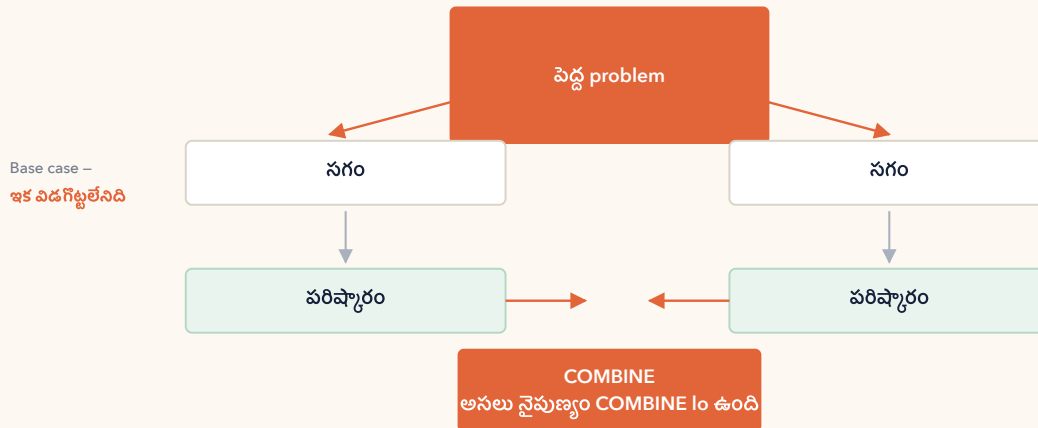
Backtracking కి తేడా ఏమిటి? Backtracking = "అన్ని possibilities enumerate చెయ్యి, dead-ends నుండి వెనక్కి రా" (search). Divide-and-Conquer = "సమస్యని ముక్కలు చేసి, ముక్కల answers కలుపు" (compute). Backtracking లో choose/un-choose ఉంటుంది; D&C లో అలా కాదు – ఒక్కసారి విడగొట్టి, కిందినుండి పైకి answers కూర్చడం.

Plain recursion కి తేడా? అన్ని D&C recursive. కానీ D&C యొక్క గుర్తు – **సమస్యని multiple subproblems గా చీల్చి, వాటి results ని combine చేయడం**. ఉదా: factorial ($n \times \text{factorial}(n-1)$) కేవలం linear recursion – ఒకే subproblem, "combine" trivial. Merge sort రెండు halves ని sort చేసి **merge** చేస్తుంది – నిజమైన divide-and-conquer. తేడా: **problem size సగానికి తగ్గడం + nontrivial combine step**.

Recursion Tree: D&C ని ఒక tree గా ఊహించు. Root = full problem. ప్రతి node రెండు children గా విడిపోతుంది (సగం sized). Leaves = base cases. Tree లో **depth = $\log n$** (ప్రతిసారి సగం అయితే), **ప్రతి level లో మొత్తం పని = $O(n)$** (అన్ని nodes కలిపి). కాబట్టి చాలా D&C algorithms **$O(n \log n)$** .

DIVIDE AND CONQUER · విడగొట్టి, పరిష్కరించి, కలపడం

DIVIDE → CONQUER → COMBINE



Divide చేయడం సులభం (సగానికి కేయడం). Conquer recursion చూసుకుంటుంది.
కానీ రెండు ఫలితాలని ఎలా కలపాలి – అదే ప్రతి problem లో వేరు.
Merge sort లో merge · Quick sort లో partition · Max subarray లో మధ్య దాటే భాగం.

Real-life Scenario

ఒక పెద్ద ఎన్నికల ఓట్ల లెక్కింపు (vote counting) ఊహించుకో. 10 లక్షల ballots ఉన్నాయి, ఒక్కడివే లెక్కించడం అసాధ్యం. ఏం చేస్తావు? **Divide:** ballots ని 10 boxes గా పంచి, 10 మంది volunteers కి ఇస్తావు. **Conquer:** ప్రతి volunteer తన box లెక్కిస్తాడు (అతను కూడా పెద్దగా ఉంటే ఇంకా పంచుకోవచ్చు – recursion!). **Combine:** అందరూ తమ counts చెప్తే, నువ్వు వాటిని కూడి total. ఒక్కడివే మొత్తం లెక్కించే బదులు, పని పంచి, subtotals కలిపావు. ఇదే divide-and-conquer.

ఇంకో analogy: **Merge sort = రెండు జట్ల playing cards ని sort చేయడం.** చేతిలో 8 cards ఉన్నాయి, sort చేయాలి. వాటిని 4-4 గా రెండు కుప్పలు చేసి, ప్రతి కుప్పని విడిగా sort చెయ్యి (recursively). ఇప్పుడు రెండు sorted కుప్పలు ఉన్నాయి – వాటిని **merge:** రెండు కుప్పల పైనున్న cards చూసి, చిన్నదాన్ని తీసి కొత్త వరుసలో పెట్టు, ఇలా. రెండు sorted lists ని ఒక sorted list గా zip చేయడమే combine step.

ఎలా గుర్తించాలి (Recognition Signals)

ఈ signals కనిపిస్తే divide-and-conquer గురించి ఆలోచించు:

- "సగం-సగం విడగొట్టగలనా?" – sorted array, linked list, range $[lo, hi]$, 2D grid ని సగానికి తగ్గించగలిగితే.
- **Sorting / merging** – merge sort, quick sort, "sort this list", "merge k lists".
- **Tree ని build చేయడం** – sorted array → balanced BST, array → segment tree, grid → quad tree. "Middle ని root చేసి, ఎడమ/కుడి recursively build".

- **"Combine two halves' answers"** – max subarray (crossing sum), closest pair of points, count inversions.
- **$O(n \log n)$ కావాలి, sorting-based** – $n \log n$ target + "divide" చేయగల structure.
- **Recurrence $T(n) = a \cdot T(n/b) + f(n)$** – subproblems గా విడగొట్టి combine చేసే ఏ problem అయినా.

Template Code (JavaScript)

D&C యొక్క **canonical skeleton** – ప్రతి problem దీని రూపమే:

```
function divideAndConquer(problem) {
  // BASE CASE – చాలా చిన్నదైతే నేరుగా solve
  if (/* problem చాలా చిన్నది (0 లేదా 1 element) */) {
    return /* trivial answer */;
  }

  // DIVIDE – సగం-సగం (లేదా k భాగాలు) గా చీల్చు
  const mid = /* మధ్య point */;
  const leftPart = /* మొదటి సగం */;
  const rightPart = /* రెండో సగం */;

  // CONQUER – ప్రతి భాగాన్నీ recursively solve
  const leftAns = divideAndConquer(leftPart);
  const rightAns = divideAndConquer(rightPart);

  // COMBINE – subproblem answers ని కలిపి final answer
  return combine(leftAns, rightAns);
}
```

గుర్తుంచుకోవాల్సినవి: (1) **base case** స్పష్టంగా – recursion ఆగే బిందువు (ఖాళీ/single element); (2) **divide** సాధారణంగా middle దగ్గర ($\text{Math.floor}((\text{lo}+\text{hi})/2)$); (3) అసలు తెలివి **combine** లో ఉంటుంది – దాన్ని correct గా రాయడమే key.

Complexity

D&C complexity ని **recurrence relation** తో లెక్కిస్తాం: $T(n) = a \cdot T(n/b) + f(n)$, ఇక్కడ a = subproblems సంఖ్య, n/b = ఒక్కో subproblem size, $f(n)$ = divide+combine cost.

- **Balanced binary split + $O(n)$ combine** → $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (merge sort, sort list, merge k lists). Recursion tree: $\log n$ levels $\times O(n)$ per level.
- **Balanced binary split + $O(1)$ combine** → $T(n) = 2T(n/2) + O(1) \rightarrow O(n)$ (sorted array → BST: ప్రతి node ఒక్కసారి touch).
- **Space:** recursion depth $\times$ per-frame → సాధారణంగా **$O(\log n)$** (balanced tree depth), plus అవసరమైన auxiliary storage.
- **Master Theorem** అనే formula ఈ recurrences ని fast గా solve చేస్తుంది; కానీ interview కి " $\log n$ levels $\times$ ప్రతి level పని ఎంత" అనే recursion-tree intuition చాలు.

8. Convert Sorted Array to Binary Search Tree (LeetCode #108) – Easy

సమస్య: Ascending order లో sort చేసిన integer array `nums` ఇస్తారు. దీన్ని ఒక **height-balanced BST** గా మార్చాలి. Height-balanced అంటే – ప్రతి node కి, ఎడమ subtree height మరియు కుడి subtree height తేడా **1 కన్నా ఎక్కువ ఉండకూడదు**. (అనేక valid answers ఉండొచ్చు; ఏదైనా balanced ఒకటి చాలు.)

Constraints: $1 \leq \text{nums.length} \leq 10^4$; `nums` strictly increasing.

ఉదాహరణ:

```
Input: nums = [-10,-3,0,5,9]
Output (one valid):
      0
     / \
   -3   9
    /   \
  -10    5
```

ఎలా ఆలోచించాలి:

BST property: ఏ node కి – ఎడమ subtree లో అన్నీ చిన్నవి, కుడి subtree లో అన్నీ పెద్దవి. Array ఇప్పటికే **sorted** కాబట్టి, "ఎడమవి చిన్నవి, కుడివి పెద్దవి" అనేది array positions లోనే ఉంది – ఎడమవైపు indices చిన్నవి, కుడివైపు పెద్దవి.

Balanced కావాలంటే? **మధ్య element ని root చేస్తే** – సగం elements ఎడమకి, సగం కుడికి → రెండు subtrees సమాన size → balanced. ఇదే insight!

కాబట్టి divide-and-conquer:

- **Divide:** array ని middle దగ్గర చీల్చు. `mid` = root.
- **Conquer:** ఎడమ half (`lo..mid-1`) ని recursively balanced BST గా → left subtree. కుడి half (`mid+1..hi`) → right subtree.
- **Combine:** root node కి ఈ రెండు subtrees ని attach.

```
[-10,-3,0,5,9] → mid index 2 = 0 → root
ఎడమ [-10,-3] → mid = -3 → root.left ; [-10] దాని left
కుడి [5,9] → mid = 5 → root.right; [9] దాని right
```

ప్రతి subtree ని array copy చేయకుండా, `[lo, hi]` indices తో represent చేస్తే space ఆదా.

Optimal Approach:

Recursion over index range `[lo, hi]`. Plan:

1. `build(lo, hi)`: ఈ range ని BST గా మార్చు.
2. Base case: `lo > hi` → ఖాళీ range → `null`.

3. `mid = Math.floor((lo + hi) / 2)` → root node `nums[mid]`.
4. `node.left = build(lo, mid-1)`, `node.right = build(mid+1, hi)`.
5. Return `node`.

Solution (JavaScript):

```
// LeetCode ఇచ్చే tree node definition
function TreeNode(val, left, right) {
  this.val = val === undefined ? 0 : val;
  this.left = left === undefined ? null : left;
  this.right = right === undefined ? null : right;
}

function sortedArrayToBST(nums) {
  function build(lo, hi) {
    // BASE CASE: ఖాళీ range → subtree లేదు
    if (lo > hi) return null;

    // DIVIDE: మధ్య element ని root చేయి → balanced
    const mid = Math.floor((lo + hi) / 2);
    const node = new TreeNode(nums[mid]);

    // CONQUER: ఎడమ/కుడి halves ని recursively build
    node.left = build(lo, mid - 1); // చిన్నవి → left subtree
    node.right = build(mid + 1, hi); // పెద్దవి → right subtree

    // COMBINE: node కి రెండు subtrees attach అయ్యాయి → return
    return node;
  }

  return build(0, nums.length - 1);
}
```

Dry Run: `nums = [-10,-3,0,5,9]` (indices 0..4)

```
build(0,4): mid=2 → node(0)
  left = build(0,1): mid=0 → node(-10)
    left = build(0,-1): lo>hi → null
    right = build(1,1): mid=1 → node(-3); left build(1,0)=null; right
build(2,1)=null → node(-3)
  → node(-10){left:null, right:-3}
  right = build(3,4): mid=3 → node(5)
    left = build(3,2) → null
    right = build(4,4): mid=4 → node(9) → node(9){null,null}
    → node(5){left:null, right:9}
  → node(0){left: (-10→-3), right: (5→9)} ✅ height-balanced
```

Complexity:

- **Time:** $O(n)$ – ప్రతి array element సరిగ్గా ఒక్క tree node అవుతుంది, ఒక్కసారే touch. Recurrence $T(n) = 2T(n/2) + O(1) \rightarrow O(n)$.

- **Space:** $O(\log n)$ – recursion depth = tree height = $\log n$ (balanced). (Output tree $O(n)$ వేరు, అది కావాల్సిందే.)

గుర్తుంచుకోవాల్సినది:

"Sorted array + balanced structure" = middle-as-root divide-and-conquer. Sorted input లో middle ని pivot/root చేసి రెండు halves ని recursively process చేయడం – ఇదే binary search యొక్క tree-building రూపం. **Combine step trivial** (కేవలం pointers attach) కాబట్టి $O(n)$. ఇదే idea: **Sorted List → BST** (#109, slow/fast తో middle కనుక్కుంటాం), **build balanced BST from any structure**. "Middle element balance ఇస్తుంది" అనేది గుర్తుంచుకో.

సాధారణ తప్పులు:

- **Base case** `lo > hi`: `lo >= hi` రాస్తే single-element range (`lo == hi`) ని miss చేసి, leaf nodes create అవును. `lo > hi` సరైనది (ఖాళీ అయినప్పుడు మాత్రమే null).
- **Mid overflow:** పెద్ద numbers లో `(lo + hi)` overflow (ఇతర భాషల్లో). JS safe కాన్ అలవాటుగా `lo + Math.floor((hi - lo) / 2)` రాయడం మంచిది.
- **Array copy చేయడం:** `nums.slice(lo, mid)` పంపితే ప్రతి level లో $O(n)$ copy → $O(n \log n)$ time + $O(n \log n)$ space. Indices (`lo, hi`) పంపితే $O(n)/O(\log n)$ – cleaner.
- **Balanced ignore:** ఏదో ఒక element (మొదటిది) ని root చేస్తే skewed tree (linked list లాంటిది) వస్తుంది – "height-balanced" fail. Middle తప్పనిసరి.

9. Sort List (LeetCode #148) – Medium

సమస్య: ఒక singly linked list యొక్క `head` ఇస్తారు. దాన్ని **ascending order** లో sort చేసి కొత్త head return చెయ్యాలి. Follow-up: **$O(n \log n)$ time మరియు $O(1)$ (constant) space** లో చెయ్యగలవా?

Constraints: nodes count $0 \leq n \leq 5 \times 10^4$; $-10^5 \leq \text{Node.val} \leq 10^5$.

ఉదాహరణ:

Input: 4 → 2 → 1 → 3 → Output: 1 → 2 → 3 → 4
Input: -1 → 5 → 3 → 4 → 0 → Output: -1 → 0 → 3 → 4 → 5

ఎలా ఆలోచించాలి:

Array అయితే `.sort()` వాడేవాళ్ళం. కానీ linked list లో **random access లేదు** – index తో నేరుగా element చేరలేం, పక్క పక్క pointers ద్వారానే కదలాలి. కాబట్టి quicksort (random pivot access కావాలి) అంత సౌకర్యం కాదు. **Merge sort** linked lists కి perfect fit – ఎందుకంటే merge step కేవలం pointers ని rewire చేయడం, extra array అవసరం లేదు. అందుకే $O(1)$ space (recursion stack తప్ప) సాధ్యం.

Merge sort = classic divide-and-conquer:

- **Divide:** list ని రెండు సగాలుగా చీల్చు. Middle ఎలా కనుక్కోవాలి? **Slow/fast pointers** – fast రెట్టింపు వేగంతో వెళ్తే, fast చివరికి చేరేసరికి slow మధ్యలో ఉంటుంది. Middle దగ్గర `next` ని `null` చేసి రెండు lists గా విడగొట్టు.
- **Conquer:** ప్రతి సగాన్నీ `sortList` తో recursively sort చెయ్యి.
- **Combine:** రెండు sorted halves ని `merge` (Merge Two Sorted Lists లాంటిది – dummy head తో).

```
4→2→1→3
divide (slow/fast) → [4→2] , [1→3]
  [4→2] → [4], [2] → merge → 2→4
  [1→3] → [1], [3] → merge → 1→3
merge(2→4 , 1→3) → 1→2→3→4 ✓
```

Optimal Approach:

Top-down merge sort. Plan:

1. Base case: `head == null || head.next == null` → 0/1 node, already sorted → return `head`.
2. **Find middle & split:** `slow=head`, `fast=head.next`; fast కదిలేదాకా slow సగం చేరుతుంది. `mid = slow.next`; `slow.next = null` (కట్).
3. Recurse: `left = sortList(head)`, `right = sortList(mid)`.
4. `return merge(left, right)`.

Solution (JavaScript):

```
function ListNode(val, next) {
  this.val = val === undefined ? 0 : val;
  this.next = next === undefined ? null : next;
}

function sortList(head) {
  // BASE CASE: 0 లేదా 1 node → already sorted
  if (head === null || head.next === null) return head;

  // DIVIDE: slow/fast తో middle కనుక్కుని రెండుగా చీల్చు
  let slow = head, fast = head.next; // fast ని ఒక అడుగు ముందు మొదలుపెడితే
  while (fast !== null && fast.next !== null) { // slow మొదటి half చివర అగుతుంది
    slow = slow.next;
    fast = fast.next.next;
  }
  const mid = slow.next; // రెండో half మొదలు
  slow.next = null;      // మొదటి half ని కట్ చేయి (రెండు separate lists)

  // CONQUER: ప్రతి half ని recursively sort
  const left = sortList(head);
  const right = sortList(mid);

  // COMBINE: రెండు sorted halves ని merge
  return merge(left, right);
}

// రెండు sorted lists ని ఒక sorted list గా zip చేయడం
```

```
function merge(a, b) {
  const dummy = new ListNode(0); // fake head – edge cases సులభం
  let tail = dummy;
  while (a !== null && b !== null) {
    if (a.val <= b.val) { tail.next = a; a = a.next; } // చిన్నదాన్ని జోడించు
    else { tail.next = b; b = b.next; }
    tail = tail.next;
  }
  tail.next = a !== null ? a : b; // మిగిలిన tail ని అతికించు
  return dummy.next;
}
```

Dry Run: 4 → 2 → 1 → 3

```
sortList(4→2→1→3):
  slow=4, fast=2 → loop: fast=2, fast.next=1 ✓ → slow=2, fast=3
                      fast=3, fast.next=null → stop
  mid = slow.next = 1→3 ; slow.next=null → left list = 4→2
  left = sortList(4→2):
    slow=4, fast=2 → loop: fast.next=null → stop. mid=2, cut → 4 | 2
    left=sortList(4)=4 ; right=sortList(2)=2 ; merge(4,2):
      b.val 2<4 → 2 ; then a=4 → 2→4
  right = sortList(1→3): → 1→3 (అలానే merge → 1→3)
  merge(2→4 , 1→3):
    1<2 → 1 ; 2<3 → 2 ; 3<4 → 3 ; a=4 left → tail.next=4
    → 1→2→3→4 ✓
```

Complexity:

- **Time:** $O(n \log n)$ – recursion tree depth $\log n$ (ప్రతిసారి half), ప్రతి level లో merge కి మొత్తం $O(n)$.
 $T(n) = 2T(n/2) + O(n)$.
- **Space:** $O(\log n)$ – recursion call stack depth (top-down). Merge step in-place (pointer rewiring), extra arrays లేవు. (కచ్చితంగా $O(1)$ space కావాలంటే bottom-up merge sort – iterative, recursion లేకుండా.)

గుర్తుంచుకోవాల్సినది:

Linked list sort = merge sort (quicksort కాదు) – ఎందుకంటే merge కేవలం pointer rewiring, random access అవసరం లేదు. **Slow/fast pointer = linked list middle కనుక్కునే master technique** (ఇది cycle detection, palindrome, "middle node" problems అన్నిటిలో వస్తుంది). ఇక్కడ `fast = head.next` గా మొదలుపెట్టడం కీలకం – even-length lists లో slow మొదటి half చివర ఆగి, clean split ఇస్తుంది. Merge Two Sorted Lists (#21) ఇక్కడ combine step గా reuse అవుతుంది – building blocks కలపడం.

సాధారణ తప్పులు:

- **slow.next = null (cut) మర్చిపోవడం:** రెండు halves ని separate చేయకపోతే, left list రెండో half లోకి కూడా extend అయి, infinite recursion / wrong result. Cut తప్పనిసరి.
- **fast = head (head.next కాకుండా):** అప్పుడు 2-node list లో slow రెండో node కి పోయి, mid=null, split తప్పు → infinite recursion. `fast = head.next` (ఒక అడుగు ముందు) safe.

- **Base case మర్చిపోవడం:** `head.next === null` check లేకపోతే single node కి కూడా split try చేసి crash/loop.
- **Merge లో dummy head వాడకపోవడం:** dummy లేకుండా head ని manually handle చేస్తే edge cases (ఏ list ఖాళీ) messy. `dummy` ఎప్పుడూ cleaner.

10. Construct Quad Tree (LeetCode #427) – Medium

సమస్య: $n \times n$ binary grid (0 లు, 1 లు; $n = 2^{\text{power}}$) ఇస్తారు. దీన్ని ఒక **Quad Tree** గా represent చెయ్యాలి. Quad Tree node కి రెండు attributes: `val` (boolean – 1 అయితే true), `isLeaf` (boolean). Internal node కి **నాలుగు children:** `topLeft`, `topRight`, `bottomLeft`, `bottomRight`.

నియమం: ఒక grid region లో అన్ని cells **ఒకే value** అయితే → అది ఒక **leaf** node (`isLeaf=true`, `val` = ఆ value). లేకపోతే → **internal** node (`isLeaf=false`), దాన్ని **నాలుగు సమాన quadrants** గా విడగొట్టి, ఒక్కొక్కదాన్ని recursively construct చేస్తాం.

Constraints: $n == 2^x$, $1 \leq n \leq 64$; cells 0 / 1.

ఉదాహరణ:

```
grid = [[0,1],
        [1,0]]
అన్నీ same కాదు → internal node.
నాలుగు 1x1 quadrants: TL=0(leaf), TR=1(leaf), BL=1(leaf), BR=0(leaf).
Output: root(isLeaf=false){ TL:leaf0, TR:leaf1, BL:leaf1, BR:leaf0 }
```

ఎలా ఆలోచించాలి:

పేరులోనే ఉంది – **Quad** tree = ఒక్కో internal node నాలుగు ముక్కలుగా విడిపోతుంది (binary tree రెండు, ఇది నాలుగు). ఇది naturally divide-and-conquer:

- **మొదట చూడు:** ఈ region (square) లో అన్ని cells ఒకటేనా? అవునైతే – simple, ఒక **leaf** (ఇక విడగొట్టనవసరం లేదు). ఇదే base case.
- **కాకపోతే (mixed):** region ని **నాలుగు సమాన quadrants** గా విడగొట్టు (top-left, top-right, bottom-left, bottom-right – ఒక్కోటి half × half). ఒక్కొక్కదాన్నీ **recursively** construct చేయి. ఆ నాలుగు subtrees ని ఒక internal node కి attach.

```
2x2 grid, mixed:
      internal
     /  |  |  \
    TL TR BL BR      ← నాలుగు 1x1 regions
    (ఒక్కోటి uniform → leaf)
```

Region ని (row, col, size) – top-left corner + side length తో represent చేస్తే, grid copy అవసరం లేదు.

Optimal Approach:

Recursion over square regions. Plan:

1. `build(r, c, size)` : (r,c) మూలగా ఉన్న `size×size` square కి Quad Tree node return.
2. **Uniform check:** ఈ square లో అన్ని cells `grid[r][c]` కి సమానమేనా scan చేయి.
3. Uniform అయితే → leaf node (`new Node(value===1, true)`).
4. లేకపోతే → `half = size/2` ; నాలుగు quadrants recursively build; internal node గా return.

Solution (JavaScript):

```
// LeetCode ఇచ్చే Quad Tree node
class Node {
  constructor(val, isLeaf, topLeft, topRight, bottomLeft, bottomRight) {
    this.val = val;
    this.isLeaf = isLeaf;
    this.topLeft = topLeft || null;
    this.topRight = topRight || null;
    this.bottomLeft = bottomLeft || null;
    this.bottomRight = bottomRight || null;
  }
}

function construct(grid) {
  function build(r, c, size) {
    // ఈ square లో అన్ని cells ఒకటేనా? (uniform check)
    const first = grid[r][c];
    let uniform = true;
    for (let i = r; i < r + size && uniform; i++) {
      for (let j = c; j < c + size; j++) {
        if (grid[i][j] !== first) { uniform = false; break; }
      }
    }

    // BASE CASE: అన్నీ same → ఒక leaf (ఇక divide అక్కర్లేదు)
    if (uniform) {
      return new Node(first === 1, true); // val boolean, isLeaf true
    }

    // DIVIDE: నాలుగు సమాన quadrants గా చీల్చు
    const half = size / 2;
    // CONQUER: ఒక్కొక్కటి recursively build
    // COMBINE: internal node కి నాలుగు subtrees attach
    return new Node(
      true, // internal node val - LeetCode దీన్ని ignore చేస్తుంది (ఎదైనా ఓకే)
      false, // isLeaf = false
      build(r, c, half), // topLeft
      build(r, c + half, half), // topRight
      build(r + half, c, half), // bottomLeft
      build(r + half, c + half, half) // bottomRight
    );
  }

  return build(0, 0, grid.length);
}
```

Dry Run: grid = [[0,1],[1,0]], build(0,0,2)

```
build(0,0,2): first=grid[0][0]=0
scan: grid[0][1]=1 ≠ 0 → uniform=false
half=1 → internal node, నాలుగు quadrants:
  TL build(0,0,1): first=0, 1x1 → uniform → leaf(val=false)
  TR build(0,1,1): first=1 → leaf(val=true)
  BL build(1,0,1): first=1 → leaf(val=true)
  BR build(1,1,1): first=0 → leaf(val=false)
→ Node(isLeaf=false){ TL:F, TR:T, BL:T, BR:F } ✓
```

Uniform grid [[1,1],[1,1]] అయితే → build(0,0,2) scan లో అన్నీ 1 → uniform → ఒకే leaf(true). నాలుగు children లేవు – ఇదే compression.

Complexity:

- **Time:** $O(n^2 \log n)$ – ప్రతి build call లో uniform check $O(\text{region size})$. Recursion tree లో $\log n$ levels, ప్రతి level లో మొత్తం cells scan $O(n^2) \rightarrow O(n^2 \log n)$. (Uniform check ని children నుండి infer చేసి $O(n^2)$ కి optimize చేయొచ్చు, కానీ ఇది clean.)
- **Space:** $O(\log n)$ recursion depth. (Output tree వేరు – worst case $O(n^2)$ nodes.)

గుర్తుంచుకోవాల్సినది:

Quad Tree = 2D divide-and-conquer, నాలుగు-way split. Binary tree రెండు halves, Quad tree నాలుగు quadrants – split factor మారుతుంది కానీ pattern అదే: "uniform అయితే base case (compress); కాకపోతే k భాగాలుగా విడగొట్టి recurse". ఇదే idea: **image compression** (uniform regions ని ఒకే node గా), **spatial indexing** (2D range queries), **k-d trees**. "Region ని (r, c, size) తో represent చేయడం" – grid recursion లో grid copy తప్పించే key trick.

సాధారణ తప్పులు:

- **Uniform check తప్పు range:** $i < r + \text{size}$, $j < c + \text{size}$ – region యొక్క సరైన సరిహద్దులు. size బదులు n వాడితే మొత్తం grid scan చేసేస్తుంది. Offset (r,c) మర్చిపోవద్దు.
- **val ని 1 గా set చేయడం:** val boolean అవ్వాలి – $\text{first} == 1$ (true/false), కేవలం first కాదు. Leaf కి ఇది ముఖ్యం.
- **Quadrant order తారుమారు:** constructor arguments order – TL, TR, BL, BR. $\text{TR} = (r, c + \text{half})$, $\text{BL} = (r + \text{half}, c)$ – row/col offsets గందరగోళం చేయొద్దు.
- **Uniform అయినా divide చేయడం:** base case లేకపోతే infinite recursion (size 1 దగ్గర ఆగినా, compression పోయి tree పెద్దదవుతుంది, wrong answer).

11. Merge k Sorted Lists (LeetCode #23) – Hard

సమస్య: k sorted linked lists array (lists) ఇస్తారు – ఒక్కోటి ascending order లో ఉంది. అన్నిటిని కలిపి ఒక sorted linked list గా merge చేసి head return చెయ్యాలి.

Constraints: $0 \leq k \leq 10^4$; ఒక్కో list length $0 \leq n$; total nodes $\leq 10^4$; $-10^4 \leq \text{Node.val} \leq 10^4$.

ఉదాహరణ:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: 1 → 1 → 2 → 3 → 4 → 4 → 5 → 6
```

```
Input: lists = [] → Output: null
```

```
Input: lists = [[]] → Output: null
```

ఎలా ఆలోచించాలి:

మనకి Merge Two Sorted Lists (#21) ఎలాగో తెలుసు – రెండు sorted lists ని $O(\text{total})$ లో zip చేయడం. ఇప్పుడు k lists. ఎలా?

Naive: ఒకదాని తర్వాత ఒకటిగా merge – result కి list1 merge, దానికి list2 merge, ... అలా. కానీ ఇది

నమ్మది: result పెద్దదవుతూ, ప్రతి merge లో దాన్ని మళ్ళీ మళ్ళీ traverse. k lists, ఒక్కటి $n \rightarrow$ merges: $n+2n+3n+\dots+kn = O(k^2n)$. k పెద్దదైతే waste.

- **Naive Time:** $O(k^2n)$ – ఒక్కో list ని ఎక్కువసార్లు traverse.
- **ఎందుకు సరిపోదు:** ముందు merge చేసిన పెద్ద result ని ప్రతిసారి మళ్ళీ scan.

Optimal – Divide and Conquer: merge sort యొక్క combine step ని గుర్తుతెచ్చుకో. k lists ని **జతలుగా** merge చేద్దాం: k lists $\rightarrow k/2$ lists (జతలు merge) $\rightarrow k/4 \rightarrow \dots \rightarrow 1$. అంటే lists array ని **సగం-సగం** గా విడగొట్టి, ఒక్కో half ని recursively merge చేసి, చివర రెండు results ని merge.

```
[L0, L1, L2, L3]
divide → [L0,L1] , [L2,L3]
[L0,L1] → merge(L0,L1) = A
[L2,L3] → merge(L2,L3) = B
merge(A, B) → final
```

ఇందులో ప్రతి node ఒక్కసారే merge అవుతుంది, కానీ merge "levels" $\log k$ మాత్రమే $\rightarrow$ ప్రతి level లో మొత్తం $O(N)$ work ($N = \text{total nodes}$) $\rightarrow$ **$O(N \log k)$** . Naive కన్నా చాలా వేగం.

Optimal Approach:

Divide-and-conquer over the `lists` array indices `[lo, hi]` . Plan:

1. Edge: `lists.length === 0` $\rightarrow$ `null` .
2. `mergeRange(lo, hi)` : `lists[lo..hi]` ని ఒక list గా merge.
3. Base: `lo === hi` $\rightarrow$ `lists[lo]` (ఒక్క list).
4. `mid` ; `left = mergeRange(lo, mid)` , `right = mergeRange(mid+1, hi)` ; `return mergeTwo(left, right)` .

Solution (JavaScript):

```

function ListNode(val, next) {
  this.val = val === undefined ? 0 : val;
  this.next = next === undefined ? null : next;
}

function mergeKLists(lists) {
  if (lists.length === 0) return null; // edge: ఏ list లేదు

  function mergeRange(lo, hi) {
    // BASE CASE: ఒక్క list మిగిలింది
    if (lo === hi) return lists[lo];

    // DIVIDE: array ని సగం-సగం
    const mid = Math.floor((lo + hi) / 2);
    // CONQUER: ఒక్క half ని recursively merge
    const left = mergeRange(lo, mid);
    const right = mergeRange(mid + 1, hi);
    // COMBINE: రెండు sorted results ని merge
    return mergeTwo(left, right);
  }

  return mergeRange(0, lists.length - 1);
}

// రెండు sorted lists ని merge (Merge Two Sorted Lists – reused)
function mergeTwo(a, b) {
  const dummy = new ListNode(0);
  let tail = dummy;
  while (a !== null && b !== null) {
    if (a.val <= b.val) { tail.next = a; a = a.next; }
    else { tail.next = b; b = b.next; }
    tail = tail.next;
  }
  tail.next = a !== null ? a : b; // మిగిలిన tail అతికించు
  return dummy.next;
}

```

Dry Run: lists = [[1,4,5], [1,3,4], [2,6]] (indices 0,1,2)

```

mergeRange(0,2): mid=1
left = mergeRange(0,1): mid=0
  left=mergeRange(0,0)=[1,4,5] ; right=mergeRange(1,1)=[1,3,4]
  mergeTwo([1,4,5], [1,3,4]):
    1≤1→1 ; 1<4→1 ; 3<4→3 ; 4≤4→4 ; 4<5→4 ; 5 left → [1,1,3,4,4,5]
right = mergeRange(2,2) = [2,6]
mergeTwo([1,1,3,4,4,5], [2,6]):
  1,1 (≤2) ; 2<3 ; 3,4,4 (<6) ; 5<6 ; 6 → [1,1,2,3,4,4,5,6] ✓

```

Complexity:

- **Time:** $O(N \log k)$ – N = total nodes అన్ని lists కలిపి, k = lists count. Divide tree depth $\log k$; ప్రతి level లో అన్ని merges కలిపి ప్రతి node ఒక్కసారి touch → $O(N)$ per level → $O(N \log k)$. (Naive $O(kN)$ కన్నా మెరుగు.)

- **Space:** $O(\log k)$ – recursion depth. Merge in-place (pointer rewiring). (Min-heap approach కూడా $O(N \log k)$ time కానీ $O(k)$ space.)

గుర్తుంచుకోవాల్సినది:

"**k things ని merge/combine చెయ్యి**" → **జతలుగా divide-and-conquer** (sequential గా ఒక్కోటి కాదు). ఇదే idea merge sort లోని merge tree – sequential $O(k^2)$ ని pairwise $O(k \log k)$ merges గా మార్చడం. **Alternative: min-heap** (k heads ని heap లో పెట్టి, చిన్నదాన్ని తీస్తూ) – అదే $O(N \log k)$, interviewer ఏది కావాలంటే అది. Merge Two Sorted Lists అనే building block ని పైన పేర్చడం (composition) గమనించు. "Combine pairwise to cut a linear chain into a log-depth tree" – ఇది reusable insight (ఉదా: k-way file merge, tournament brackets).

సాధారణ తప్పులు:

- **Empty input:** `lists = []` కి `mergeRange(0, -1)` పంపితే bug. ముందే `if (lists.length === 0) return null`.
- **lists = [[]]:** ఒక ఖాళీ list ఉంది → `mergeRange(0, 0)` returns `lists[0] = null` → `mergeTwo(null, ...)` handle చేయాలి. `mergeTwo` లో `while (a && b) + tail.next = a ?? b` వల్ల null-safe.
- **Sequential merge (naive) వాడటం:** correct కానీ $O(k^2n)$ – interview లో "optimize చెయ్యి" అంటే pairwise D&C లేదా heap.
- **mid+1 off-by-one:** `right = mergeRange(mid+1, hi)` – `mid` కాదు, `mid+1`. `mergeRange(mid, hi)` పంపితే overlap/infinite recursion.
- **Merge లో <= vs <:** duplicate values కి stability కి <= వాడు (ఇక్కడ order మారదు కానీ practice).

సారాంశం (Cheat Sheet – interview కి ముందు ఒక్క చూపు)

Backtracking – ఒక్క మంత్రం: choose → explore → un-choose. `path.push` ఉన్నచోట `path.pop` ఉండాలి. Leaf దగ్గర `result.push([...path])` (copy!).

అవసరం	వాడాల్సినది
Order పట్టింపు లేదు (combinations, subsets)	<code>start</code> pointer, <code>backtrack(i+1)</code>
Order పట్టింపు ఉంది (permutations)	<code>used[]</code> array, ప్రతిసారి 0 నుండి
ఒక element reuse allowed (combination sum)	<code>backtrack(i)</code> (i+1 కాదు)
Duplicates ఉన్న input	sort + <code>if (i>start && a[i]===a[i-1]) continue</code>
Board/grid (word search, N-Queens)	DFS + visited mark/un-mark; diagonal = <code>row±col</code>
Constraints తో string build (parentheses)	validity rules నే branching conditions గా

Divide and Conquer – ఒక్క మంత్రం: divide (సగం) → conquer (recurse) → combine. అసలు తెలివి combine లో.

Problem	Divide	Combine	Complexity
Sorted array → BST	middle = root	pointers attach	$O(n)$
Sort List	slow/fast middle	merge two lists	$O(n \log n)$
Quad Tree	4 quadrants	4 children attach	$O(n^2 \log n)$
Merge k Lists	సగం lists array	merge two lists	$O(N \log k)$

రెండింటి తేడా: Backtracking = అన్ని possibilities వెతకడం (choose/un-choose తో). Divide-and-Conquer = ముక్కల answers ని కూర్చడం (choose/un-choose లేదు). Complexity: backtracking సాధారణంగా exponential (tree లో nodes count), D&C సాధారణంగా $O(n \log n)$ ($\log n$ levels $\times O(n)$ పని).

చివరి మాట: ఈ 11 problems బట్టి కొట్టొద్దు. **రెండు templates** (backtracking skeleton, D&C skeleton) చేతిలో పెట్టుకో. కొత్త problem వచ్చినప్పుడు – "ఇది అన్ని possibilities వెతకాలా (backtracking), లేక ముక్కలుగా విడగొట్టి కూర్చాలా (D&C)?" అని అడుగు. Answer దానంతట అదే బయటకు వస్తుంది. అదే "ఒకసారి చదివితే మర్చిపోకూడదు."